

Extended thinking

Downloads

001_thinking_complete.ipynb

(opens in new tab)

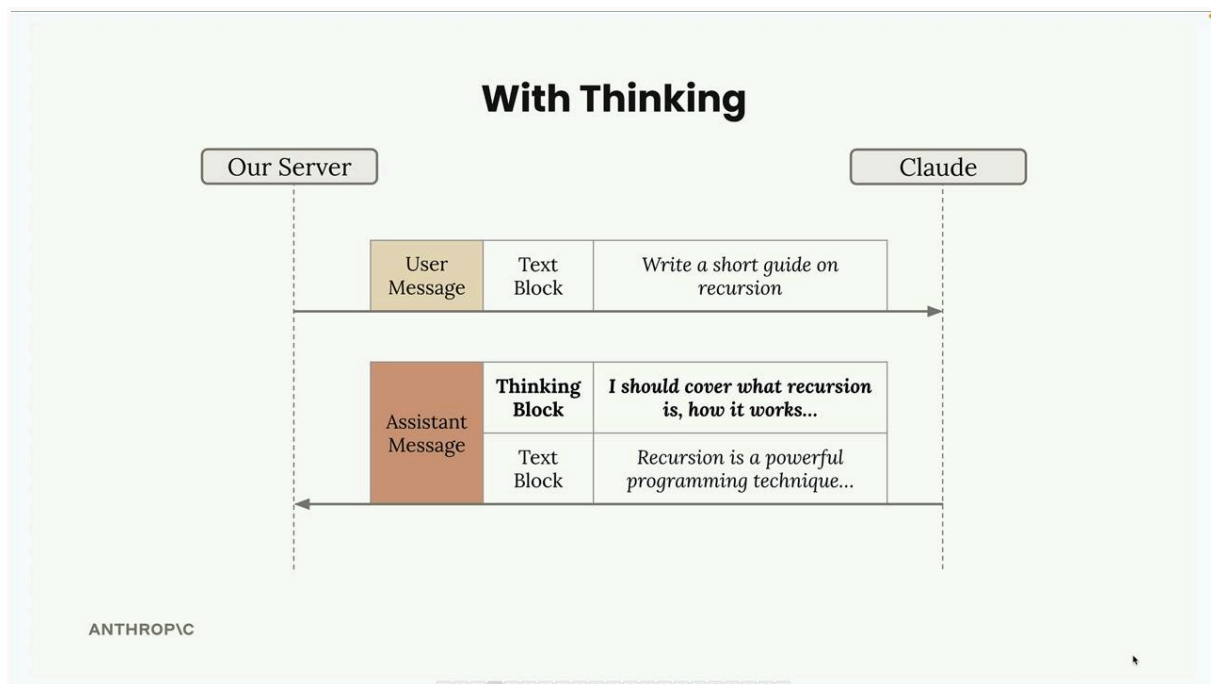
001_thinking.ipynb

Important Note: Extended Thinking is not compatible with some other features, notable message pre-filling and temperature. See the full list of restrictions here:
<https://docs.anthropic.com/en/docs/build-with-claude/extended-thinking#feature-compatibility>

Extended thinking is Claude's advanced reasoning feature that gives the model time to work through complex problems before generating a final response. Think of it as Claude's "scratch paper" - you can see the reasoning process that leads to the answer, which helps with transparency and often results in better quality responses.

How Extended Thinking Works

When extended thinking is enabled, Claude's response changes from a simple text block to a structured response containing two parts:



With thinking enabled, you get both the reasoning process and the final answer:

Assistant Message

The Signature

- Cryptographic token tied to the thinking text that Claude generated
- Verifies that the exact text was generated by Claude and was not changed

```

{
  "content": [
    {
      "type": "thinking",
      "thinking": "The user is asking me to...",
      "signature": "EqoBCkgIAhABGAIiQDbvHj8yA..."
    },
    {
      "type": "text",
      "text": "A Guide to Recursion\n\n"
    }
  ]
}

```

ANTHROPIC

The key benefits include:

- Better reasoning capabilities for complex tasks

- Increased accuracy on difficult problems
- Transparency into Claude's thought process

However, there are important trade-offs:

- Higher costs (you pay for thinking tokens)
- Increased latency (thinking takes time)
- More complex response handling in your code

When to Use Extended Thinking

The decision is straightforward: use your prompt evaluations. Run your prompts without thinking first, and if the accuracy isn't meeting your requirements after you've already optimized your prompt, then consider enabling extended thinking. It's a tool for when standard prompting isn't quite getting you there.

Response Structure and Security

Extended thinking responses include a special signature system for security:

The Signature

- Cryptographic token tied to the thinking text that Claude generated
- Verifies that the exact text was generated by Claude and was not changed

Assistant Message

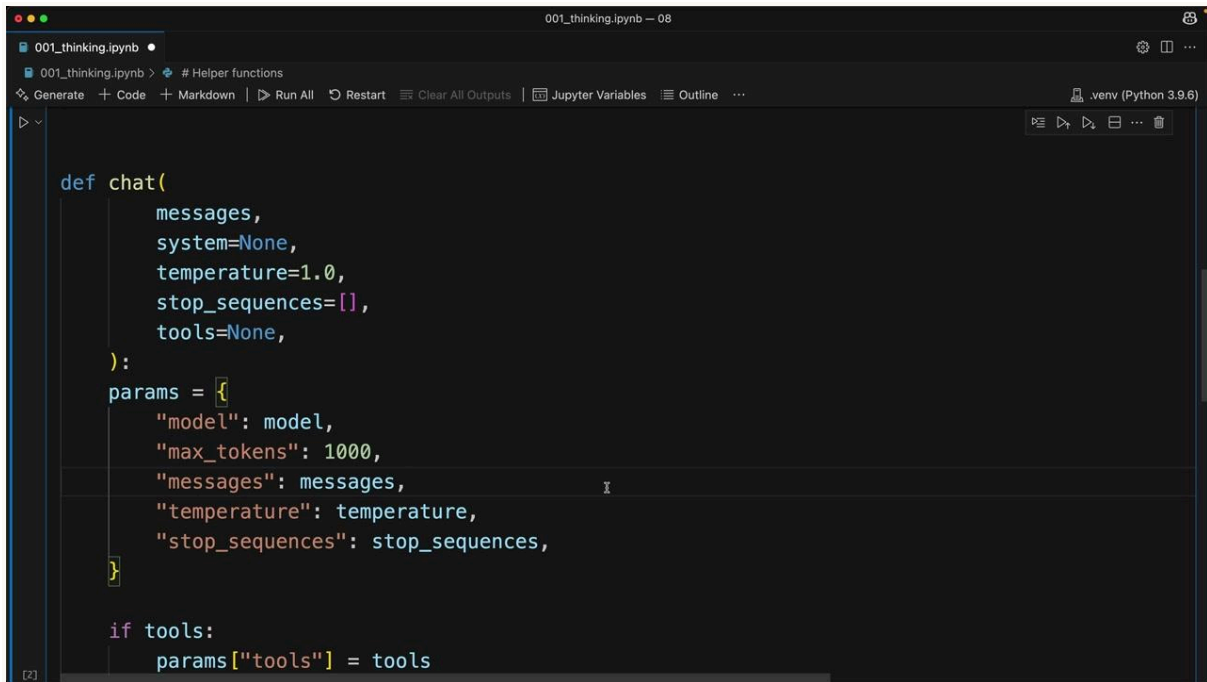
```
{
  "content": [
    {
      "type": "thinking",
      "thinking": "The user is asking me to...",
      "signature": "EqoBCkgIAhABGAIiQDbvHj8yA..."
    },
    {
      "type": "text",
      "text": "A Guide to Recursion\n\n"
    }
  ]
}
```

ANTHROPIC

The signature is a cryptographic token that ensures you haven't modified the thinking text. This prevents developers from tampering with Claude's reasoning process, which could potentially lead the model in unsafe directions.

Redacted Thinking

Sometimes you'll receive a redacted thinking block instead of readable reasoning text:

A screenshot of a Jupyter Notebook window titled '001_thinking.ipynb - 08'. The interface shows a code editor with a Python function definition. The function 'chat' takes 'messages', 'system', 'temperature', 'stop_sequences', and 'tools' as arguments. It constructs a 'params' dictionary with keys for 'model', 'max_tokens', 'messages', 'temperature', and 'stop_sequences'. If 'tools' is provided, it adds 'tools' to the dictionary. The notebook interface includes a top bar with file and view icons, a toolbar with actions like 'Generate', 'Code', 'Markdown', 'Run All', 'Restart', 'Clear All Outputs', 'Jupyter Variables', and 'Outline', and a status bar indicating the environment is '.venv (Python 3.9.6)'.

This happens when Claude's thinking process gets flagged by internal safety systems. The redacted content contains the actual thinking in encrypted form, allowing you to pass the complete message back to Claude in future conversations without losing context.

Implementation

To enable extended thinking in your code, you need to add two parameters to your chat function:

```
def chat(  
    messages,  
    system=None,  
    temperature=1.0,  
    stop_sequences=[],  
    tools=None,
```

```
thinking=False,  
thinking_budget=1024
```

):

The thinking budget sets the maximum tokens Claude can use for reasoning. The minimum value is 1024 tokens, and your **max_tokens** parameter must be greater than your thinking budget.

Add the thinking configuration to your API parameters:

```
if thinking:  
    params["thinking"] = {  
        "type": "enabled",  
        "budget": thinking_budget  
    }
```

Then call it with thinking enabled:

```
chat(messages, thinking=True)
```

Testing Redacted Responses

For testing purposes, you can force Claude to return a redacted thinking block by sending a special trigger string. This helps ensure your application handles redacted responses gracefully without crashing.

Extended thinking is a powerful feature when you need Claude to tackle complex reasoning tasks, but use it judiciously given the cost and latency implications. Start with standard prompting, optimize

thoroughly, then add thinking when you need that extra reasoning capability.

Image support

Downloads

images.zip

(opens in new tab)

002_images.ipynb

Claude's vision capabilities let you include images in your messages and ask Claude to analyze them in countless ways. You can ask Claude to describe what's in an image, compare multiple images, count objects, or perform complex visual analysis tasks.

Image Handling Basics

Image Handling

- Up to 100 images across all messages in a single request
- Max size of 5MB
- When sending **one** image max height/width of 8000px
- When sending **multiple** images, max height/width of 2000px
- Images can be included in base64 encoding, or a URL to the image
- Each image counts as a certain number of tokens:
$$\text{tokens} = \frac{(\text{width px} \times \text{height px})}{750}$$

ANTHROPIC

```
with open("image.png", "rb") as f:
    image_bytes = base64.standard_b64encode(
        f.read()
    ).decode("utf-8")

add_user_message(messages, [
    # Image Block
    {
        "type": "image",
        "source": {
            "type": "base64",
            "media_type": "image/png",
            "data": image_bytes,
        }
    },
    # Text Block
    {
        "type": "text"
        "text": "What do you see in this image?"
    }
])
```

There are several important limitations to keep in mind when working with images:

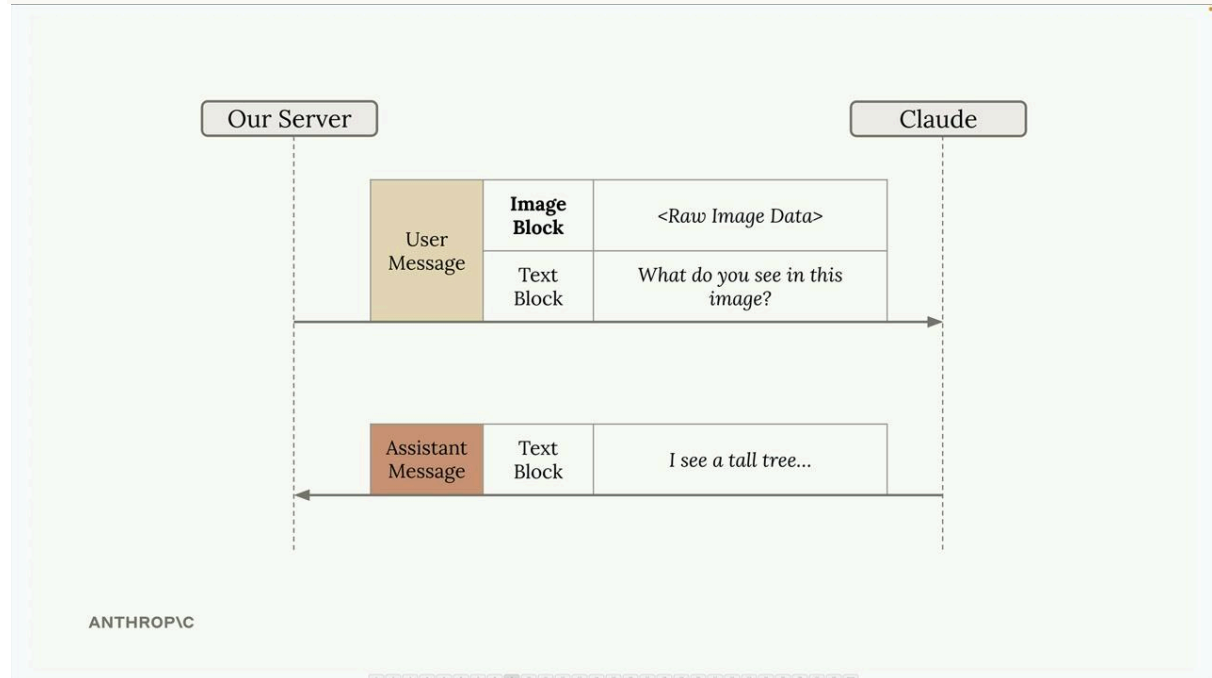
- Up to 100 images across all messages in a single request
- Max size of 5MB per image
- When sending one image: max height/width of 8000px
- When sending multiple images: max height/width of 2000px
- Images can be included as base64 encoding or a URL to the image
- Each image counts as tokens based on its dimensions: **tokens = (width px × height px) / 750**

To send an image to Claude, you include an image block in your user message alongside text blocks. Here's the structure:

```
with open("image.png", "rb") as f:
    image_bytes =
base64.standard_b64encode(f.read()).decode("utf-8")

add_user_message(messages, [
    # Image Block
    {
        "type": "image",
        "source": {
            "type": "base64",
            "media_type": "image/png",
            "data": image_bytes,
        }
    },
    # Text Block
    {
        "type": "text",
        "text": "What do you see in this image?"
    }
])
```

Message Flow

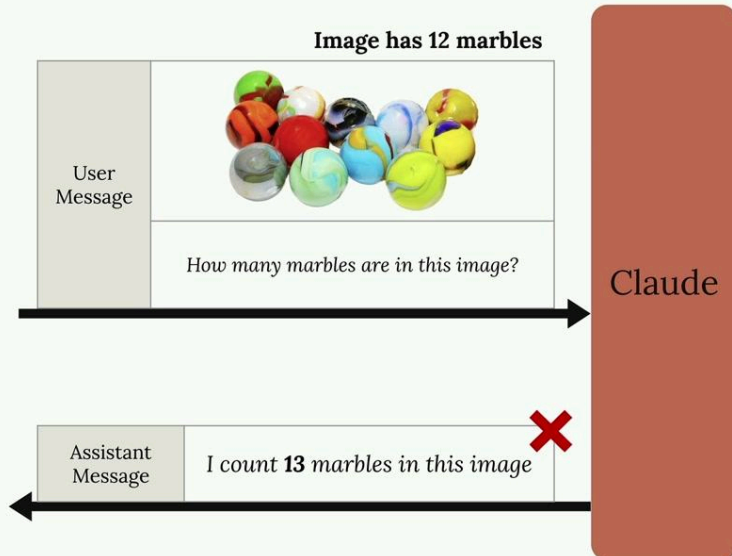


The conversation works just like text-only interactions. Your server sends a user message containing both image and text blocks to Claude, and Claude responds with a text block containing its analysis.

Prompting Techniques

Prompting Techniques

- All the same prompting engineering techniques apply to images
- You can increase Claude's vision accuracy by providing guidelines, analysis steps, or by using one-shot/multi-shot examples



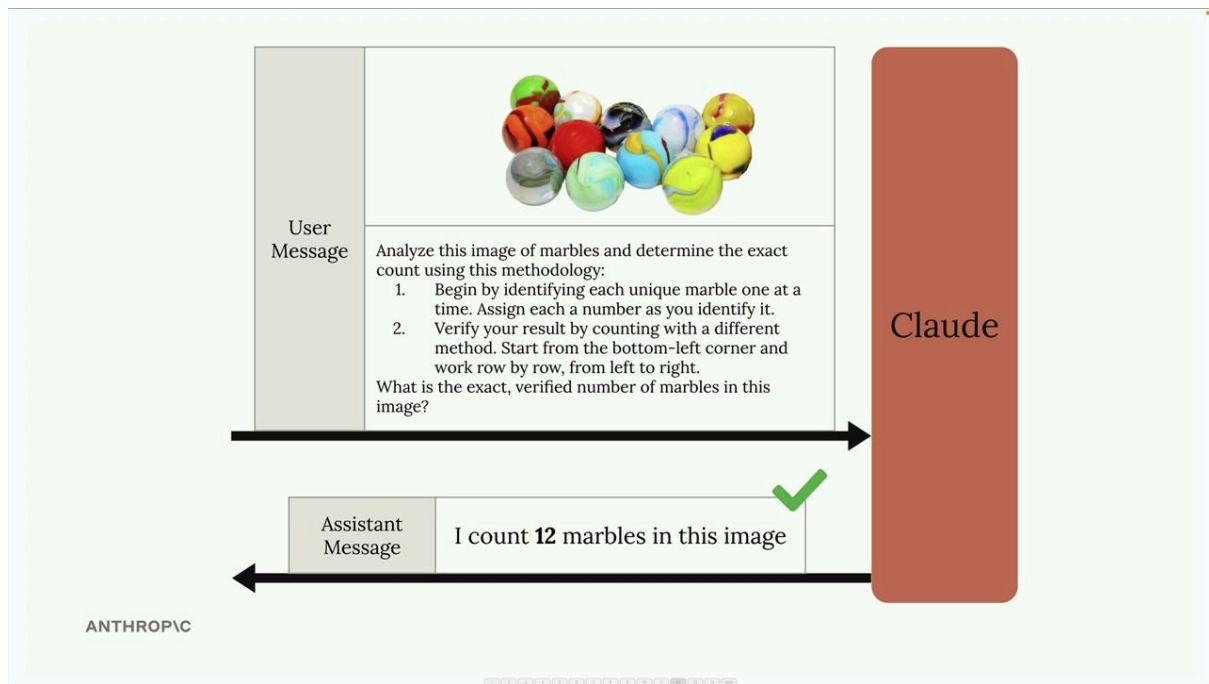
ANTHROPIC

The key to getting good results with images is applying the same prompting engineering techniques you'd use with text. Simple prompts often lead to poor results. For example, asking "How many marbles are in this image?" might return an incorrect count.

You can dramatically improve Claude's accuracy by:

- Providing detailed guidelines and analysis steps
- Using one-shot or multi-shot examples
- Breaking down complex tasks into smaller steps

Step-by-Step Analysis



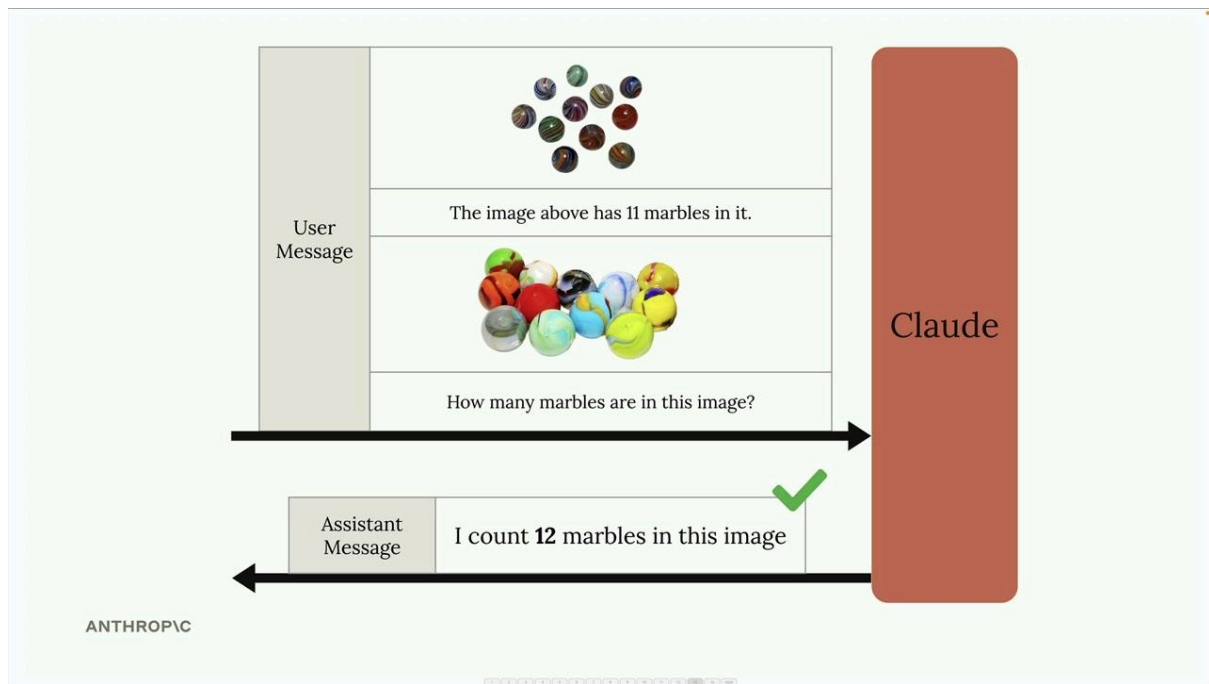
Instead of a simple question, provide Claude with a methodology:

Analyze this image of marbles and determine the exact count using this methodology:

1. Begin by identifying each unique marble one at a time. Assign each a number as you identify it.
2. Verify your result by counting with a different method. Start from the bottom-left corner and work row by row, from left to right.

What is the exact, verified number of marbles in this image?

One-Shot Examples



You can also improve accuracy by providing examples within your message. Include an image with a known count, state the correct answer, then ask about your target image. This gives Claude a reference point for the type of analysis you want.

Real-World Example: Fire Risk Assessment

Fire Risk Assessments

- In some parts of the US, home fire insurance companies requires homeowners to trim or remove trees around the residence
- Sending an inspector out to each property would be expensive
- **Solution:** get high resolution, up-to-date satellite imagery, ask Claude for a fire risk assessment

ANTHROPIC



Here's a practical application: automating fire risk assessments for home insurance. Instead of sending inspectors to every property, insurance companies can use satellite imagery and Claude's analysis.

The system analyzes satellite images to identify:

- Dense, close-packed trees near the residence
- Difficult access routes for emergency services
- Branches overhanging the residence

Rather than a simple prompt like "provide a fire risk score," a well-structured prompt breaks down the analysis into specific steps:

Analyze the attached satellite image of a property with these specific steps:

1. Residence identification: Locate the primary residence on the property by looking for:

- The largest roofed structure
- Typical residential features (driveway connection, regular geometry)
- Distinction from other structures (garages, sheds, pools)

2. Tree overhang analysis: Examine all trees near the primary residence:

- Identify any trees whose canopy extends directly over any portion of the roof
- Estimate the percentage of roof covered by overhanging branches (0-25%, 25-50%, 50-75%, 75%+)
- Note particularly dense areas of overhang

3. Fire risk assessment: For any overhanging trees, evaluate:

- Potential wildfire vulnerability (ember catch points, continuous fuel paths to structure)
- Proximity to chimneys, vents, or other roof openings if visible
- Areas where branches create a "bridge" between wildland vegetation and the structure

4. Defensible space identification: Assess the property's overall vegetative structure:

- Identify if trees connect to form a continuous canopy over or near the home
- Note any obvious fuel ladders (vegetation that can carry fire from ground to tree to roof)

5. Fire risk rating: Based on your analysis, assign a Fire Risk Rating from 1-4:

- Rating 1 (Low Risk): No tree branches overhanging the roof, good defensible space around the home
- Rating 2 (Moderate Risk): Minimal overhang (<25% of roof), some separation between tree canopies
- Rating 3 (High Risk): Significant overhang (25-50% of roof), connected tree canopies, multiple vulnerability points
- Rating 4 (Severe Risk): Extensive overhang (>50% of roof), dense vegetation against structure

For each item above (1-5), write one sentence summarizing your findings, with your final response being the numerical rating.

This detailed prompt guides Claude through a systematic analysis, resulting in much more accurate and useful assessments than a simple request would provide.

Remember: the same prompting techniques that work for text apply to images. Invest time in crafting detailed, structured prompts rather than relying on simple questions if you want reliable results.

PDF support

Downloads

earth.pdf

Claude can read and analyze PDF files directly, making it a powerful tool for document processing. This capability works similarly to image processing, but with a few key differences in how you structure your code.

Setting Up PDF Processing

To process a PDF file with Claude, you'll use nearly identical code to what you'd use for images. The main differences are in the file type specifications and variable names for clarity.

Here's how to modify your existing image processing code for PDFs:

```
with open("earth.pdf", "rb") as f:
    file_bytes =
base64.standard_b64encode(f.read()).decode("utf-8")

messages = []

add_user_message(
    messages,
    [
        {
            "type": "document",
            "source": {
                "type": "base64",
                "media_type": "application/pdf",
                "data": file_bytes,
            },
        },
    ],
)
```

```
    },  
    {"type": "text", "text": "Summarize the document in  
one sentence"},  
  ],  
)  
  
chat(messages)
```

Key Changes from Image Processing

When adapting your image processing code for PDFs, you need to update several elements:

- Change the file extension from **.png** to **.pdf**
- Update the variable name from **image_bytes** to **file_bytes** for clarity
- Set the type to **"document"** instead of **"image"**
- Change the media type to **"application/pdf"** instead of **"image/png"**

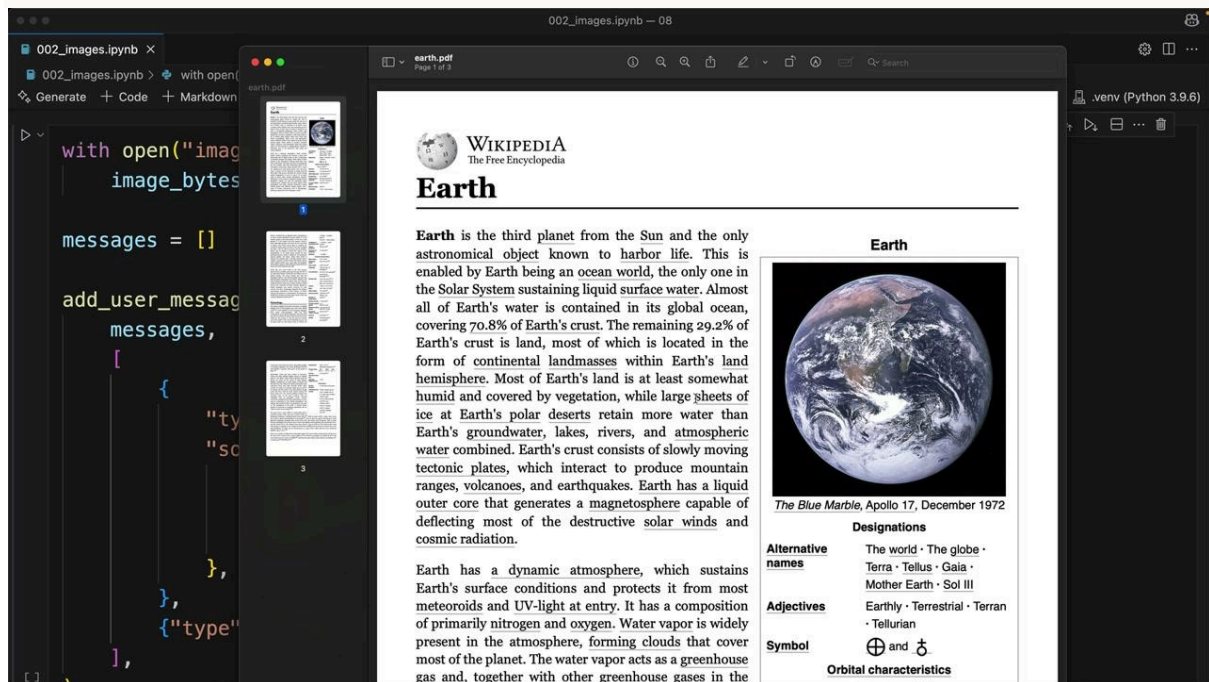
What Claude Can Extract from PDFs

Claude's PDF processing capabilities go beyond simple text extraction. It can analyze and understand:

- Text content throughout the document
- Images and charts embedded in the PDF

- Tables and their data relationships
- Document structure and formatting

This makes Claude essentially a one-stop solution for extracting any type of information from PDF documents, whether you need summaries, data analysis, or specific content extraction.



The example above shows Claude successfully processing a Wikipedia article about Earth that was saved as a PDF, demonstrating how it can understand and summarize complex document content in a single sentence.

Citations

Downloads

[002_citations_complete.ipynb](#)

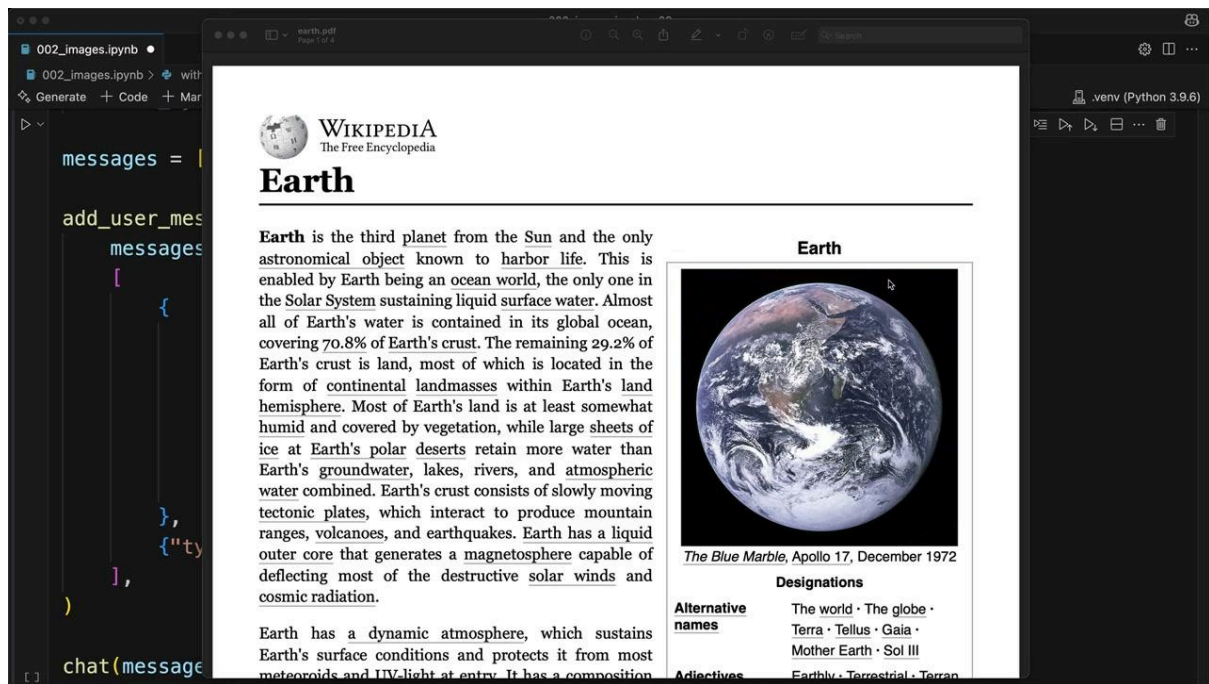
(opens in new tab)

[earth.pdf](#)

When Claude answers questions based on documents you provide, users might assume it's just drawing from its training data. But what if Claude could show exactly where it found specific information? That's where citations come in - a powerful feature that lets Claude reference specific parts of your source documents and show users exactly where each piece of information comes from.

Why Citations Matter

Imagine asking Claude about how Earth's atmosphere formed and getting a detailed answer. Without citations, users have no way to verify the information or understand that Claude is actually referencing a specific document you provided. Citations solve this transparency problem by creating a clear trail from Claude's response back to your source material.



Enabling Citations

To enable citations, you need to modify your document message structure. Add two new fields to your document block:

```
{  
  "type": "document",  
  "source": {  
    "type": "base64",  
    "media_type": "application/pdf",  
    "data": file_bytes,  
  },  
  "title": "earth.pdf",  
  "citations": { "enabled": True }  
}
```

The **title** field gives your document a readable name, while **citations: {"enabled": True}** tells Claude to track where it finds information.

Understanding Citation Structure

When citations are enabled, Claude's response becomes more complex. Instead of simple text, you get structured data that includes citation information for each claim.

Citation Structure		
	Example	Purpose
cited_text	Earth's atmosphere and oceans were formed by volcanic activity and outgassing.	Text that Claude is citing from the provided document
document_index	0	If multiple documents are provided, this will tell you which document Claude is citing from
document_title	'earth.pdf'	Title of the document that Claude is citing from
start_page_number	4	Starting page of the cited text
end_page_number	5	Ending page of the cited text

ANTHROPIC

Each citation contains several key pieces of information:

- **cited_text** - The exact text from your document that supports Claude's statement
- **document_index** - Which document Claude is referencing (useful when you provide multiple documents)

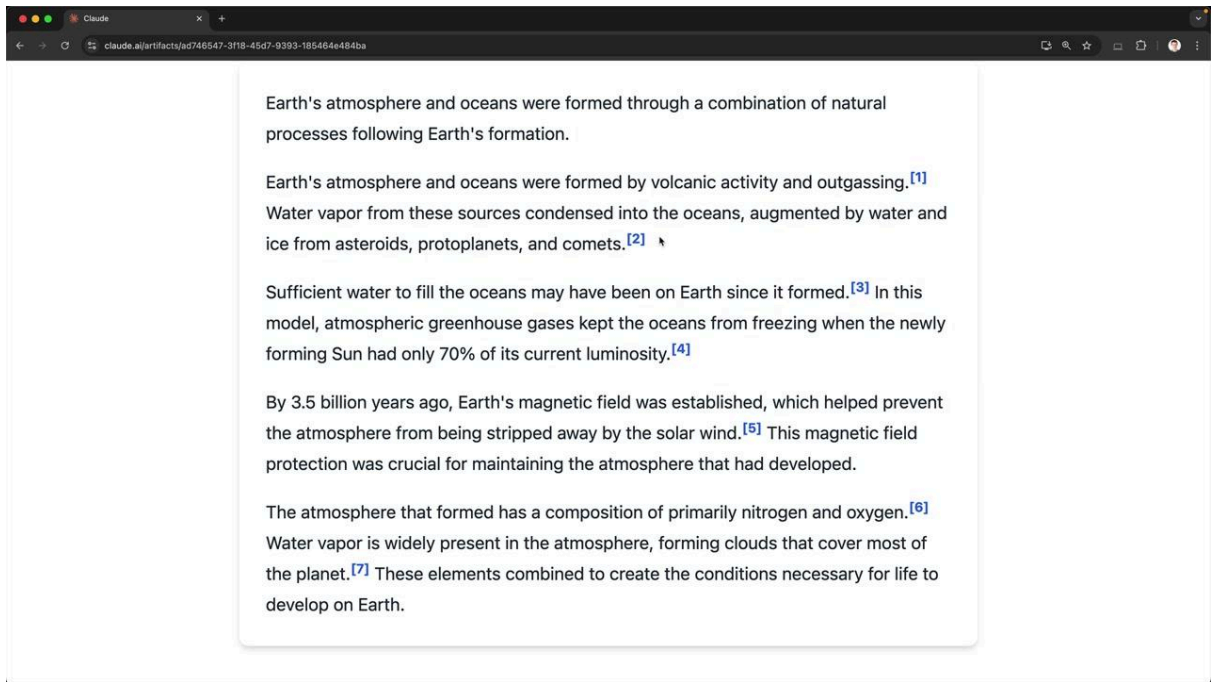
- `document_title` - The title you assigned to the document
- `start_page_number` - Where the cited text begins
- `end_page_number` - Where the cited text ends

Citation Structure		
	Example	Purpose
<code>cited_text</code>	Earth's atmosphere and oceans were formed by volcanic activity and outgassing.	Text that Claude is citing from the provided document
<code>document_index</code>	0	If multiple documents are provided, this will tell you which document Claude is citing from
<code>document_title</code>	'earth.pdf'	Title of the document that Claude is citing from
<code>start_page_number</code>	4	Starting page of the cited text
<code>end_page_number</code>	5	Ending page of the cited text

ANTHROPIC

Building User Interfaces with Citations

The real power of citations comes from building user interfaces that make this information accessible. You can create interactive elements where users can hover over citation markers to see exactly where information came from.



This creates a transparent experience where users can:

- See that Claude's answers are grounded in actual source material
- Verify the information by checking the original document
- Understand the context around each cited piece of information

Citations with Plain Text

Citations aren't limited to PDF documents. You can also use them with plain text sources. When working with text, modify your document structure like this:

```
{  
  "type": "document",  
  "source": {  
    "type": "text",  
    "media_type": "text/plain",
```

```
    "data": article_text,  
  },  
  "title": "earth_article",  
  "citations": { "enabled": True }  
}
```

With plain text sources, instead of page numbers, you'll get character positions that pinpoint exactly where in the text Claude found each piece of information.

When to Use Citations

Citations are particularly valuable when:

- Users need to verify information for accuracy
- You're working with authoritative documents that users should be able to reference
- Transparency about information sources is critical for your application
- Users might want to explore the broader context around specific facts

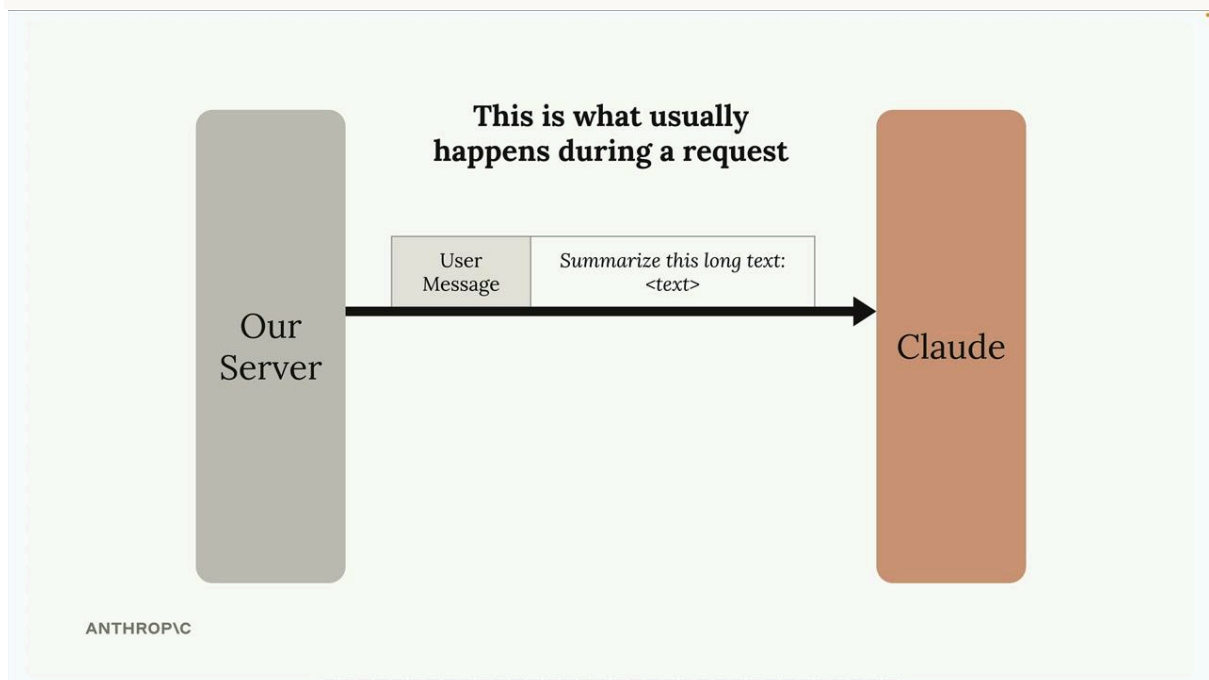
By implementing citations, you transform Claude from a "black box" that provides answers into a transparent research assistant that shows its work. This builds user trust and enables them to dive deeper into your source materials when needed.

Prompt caching

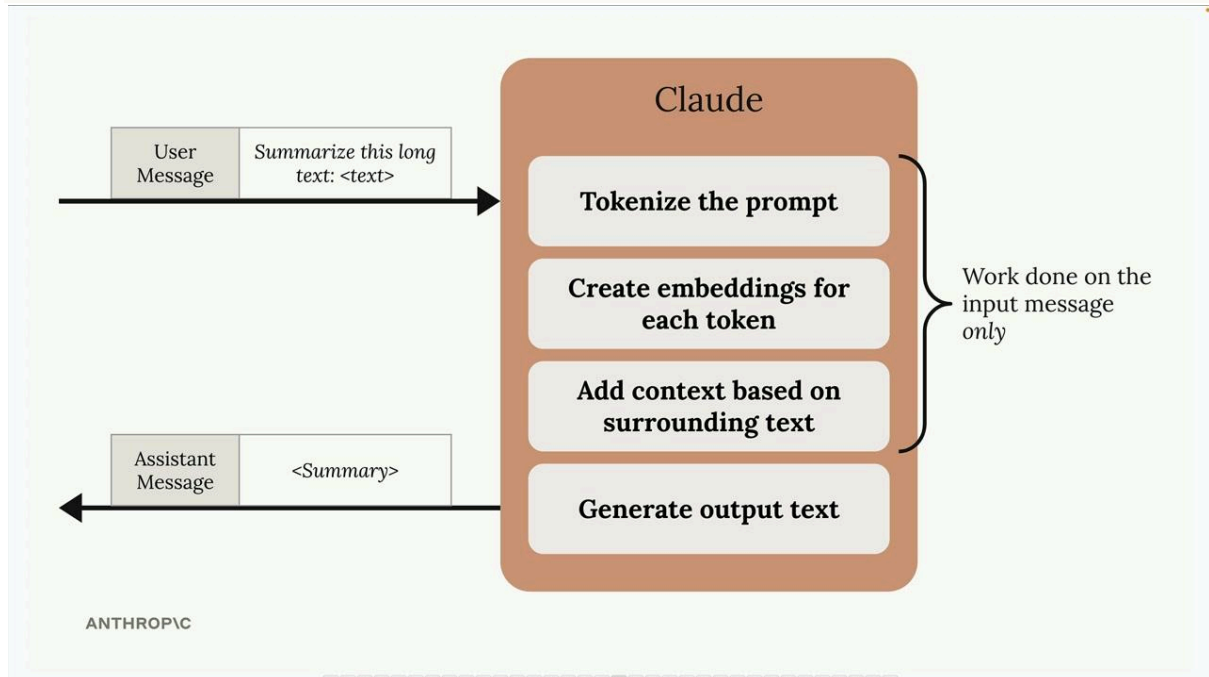
Prompt caching is a feature that speeds up Claude's responses and reduces the cost of text generation by reusing computational work from previous requests. Instead of throwing away all the processing work after each request, Claude can save and reuse it when you send similar content again.

How Claude Normally Processes Requests

To understand prompt caching, let's first look at what happens during a typical request without caching enabled.

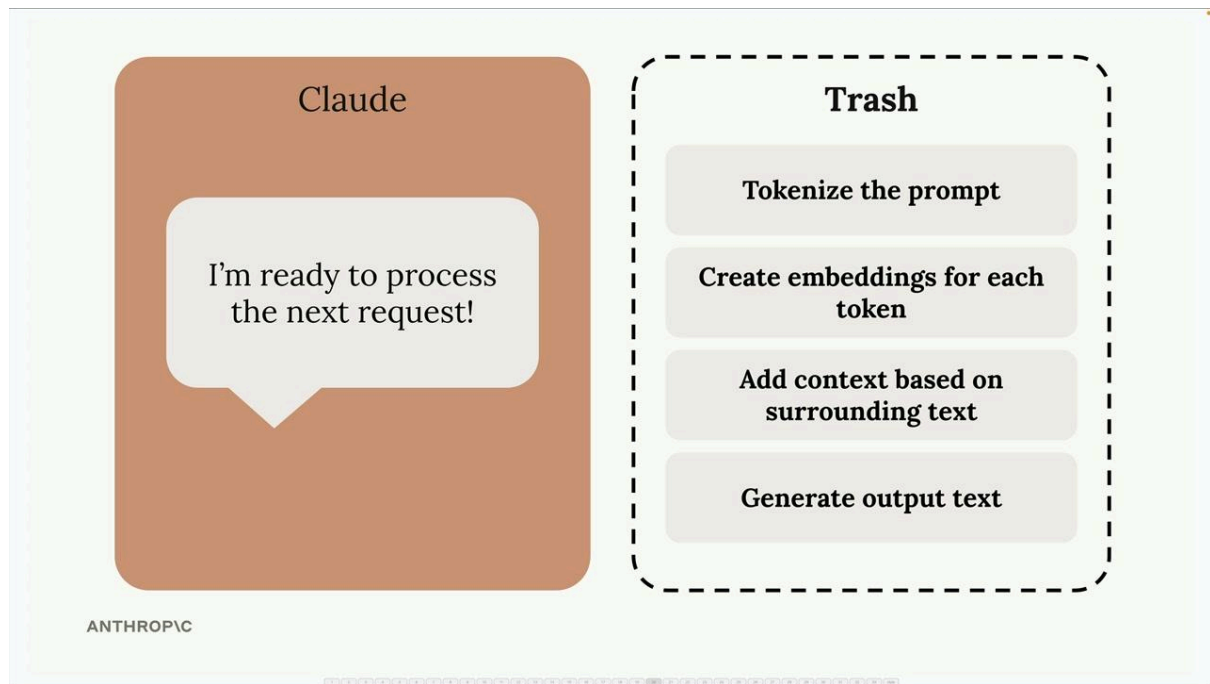


When you send a message to Claude, it doesn't immediately start generating a response. Instead, Claude does a tremendous amount of preprocessing work on your input:



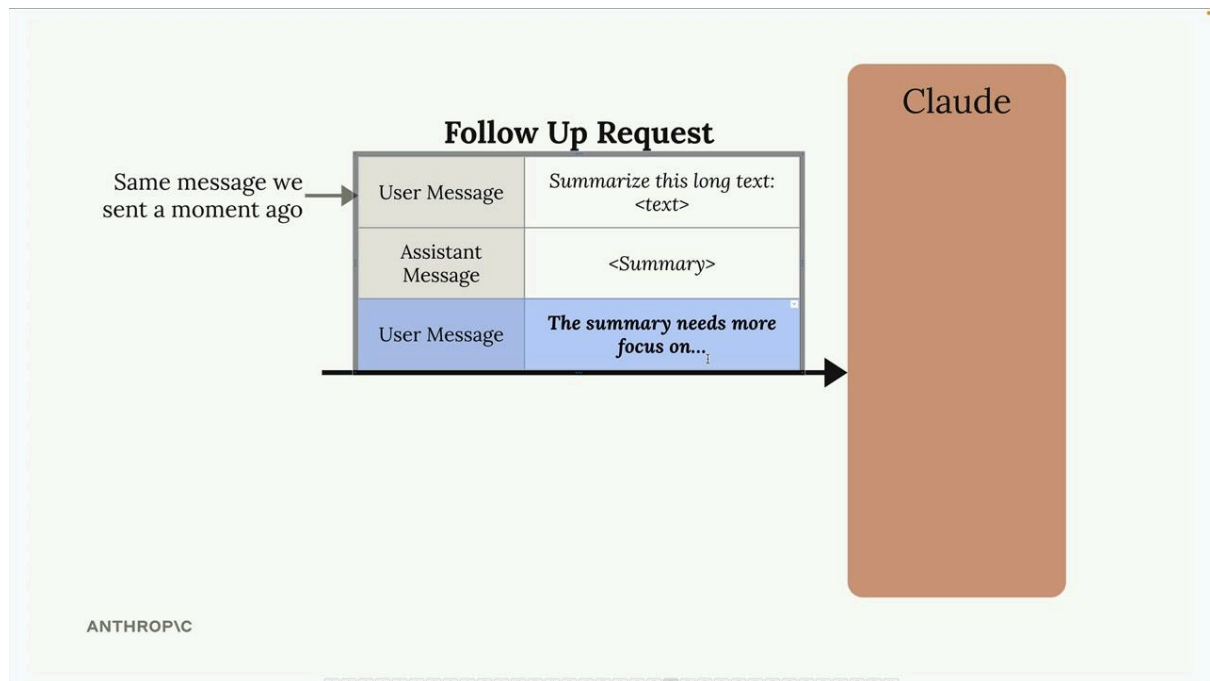
- Tokenizes the prompt into smaller pieces
- Creates embeddings for each token
- Adds context based on surrounding text
- Only then generates the actual output text

After sending you the response, Claude throws away all this computational work - the tokenization, embeddings, and context analysis all get discarded.

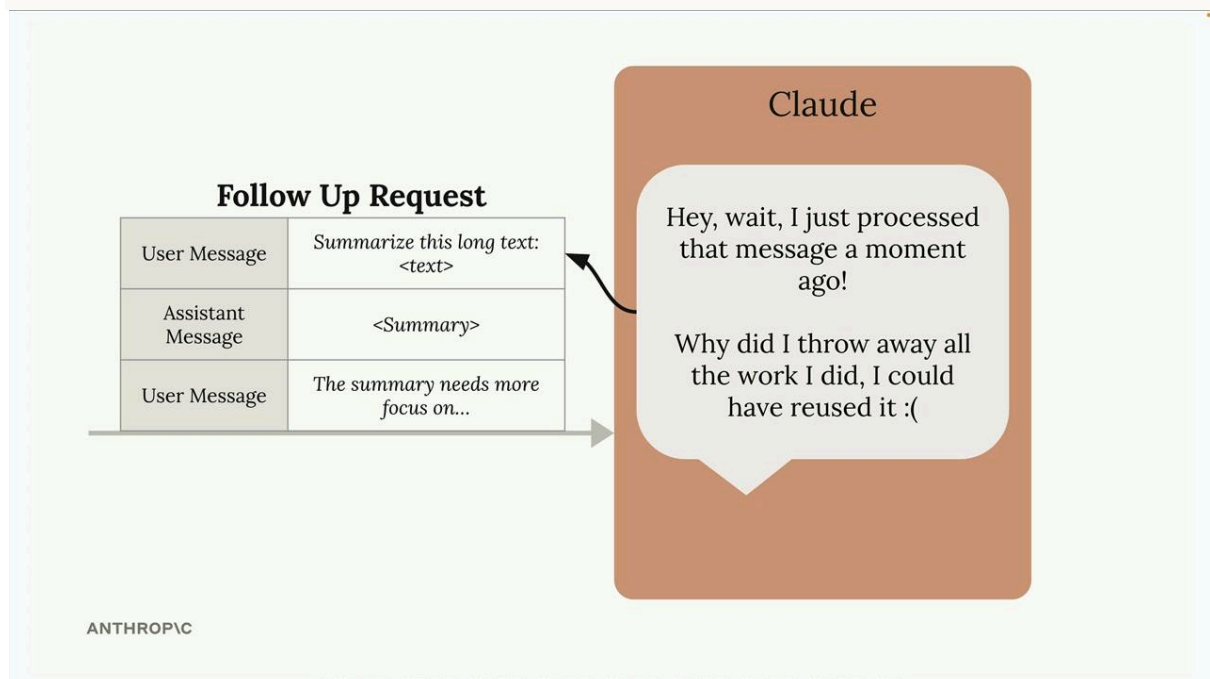


The Problem with Discarding Work

This becomes inefficient when you make follow-up requests that include the same content. For example, in a conversation where you're asking Claude to refine a summary of the same long text:

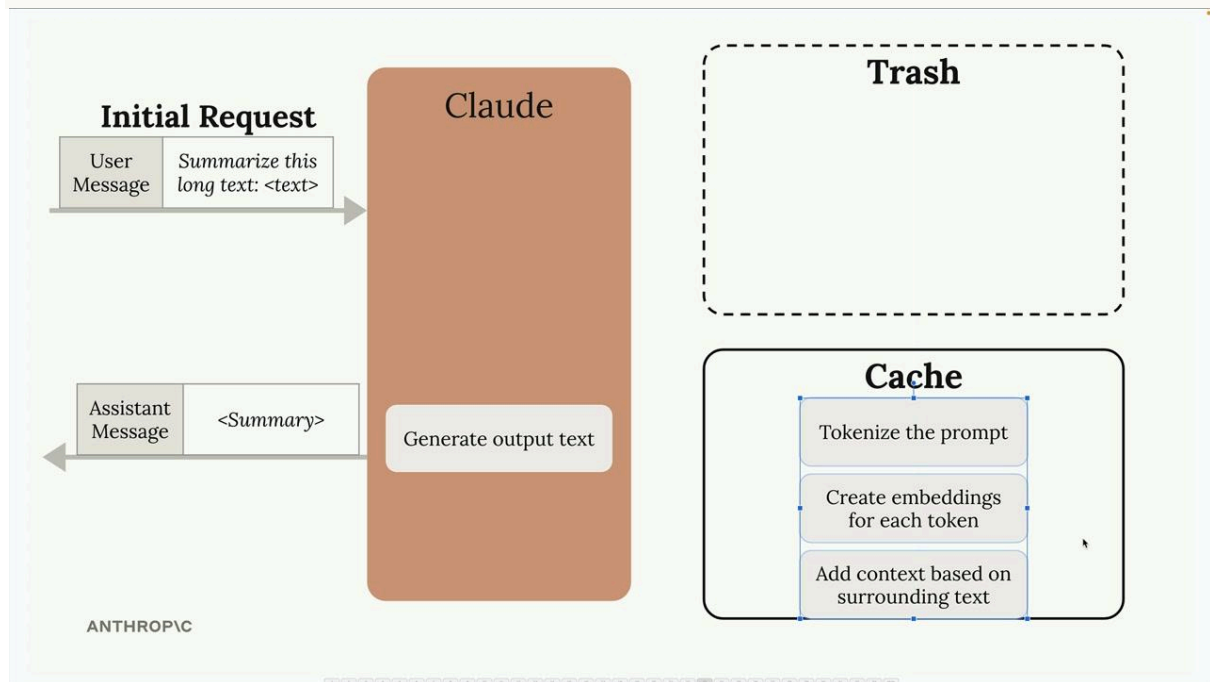


Claude has to repeat all the same preprocessing work on content it just analyzed moments ago. As Claude might think to itself: "I just processed that message and threw away all the work I did - I could have reused it!"

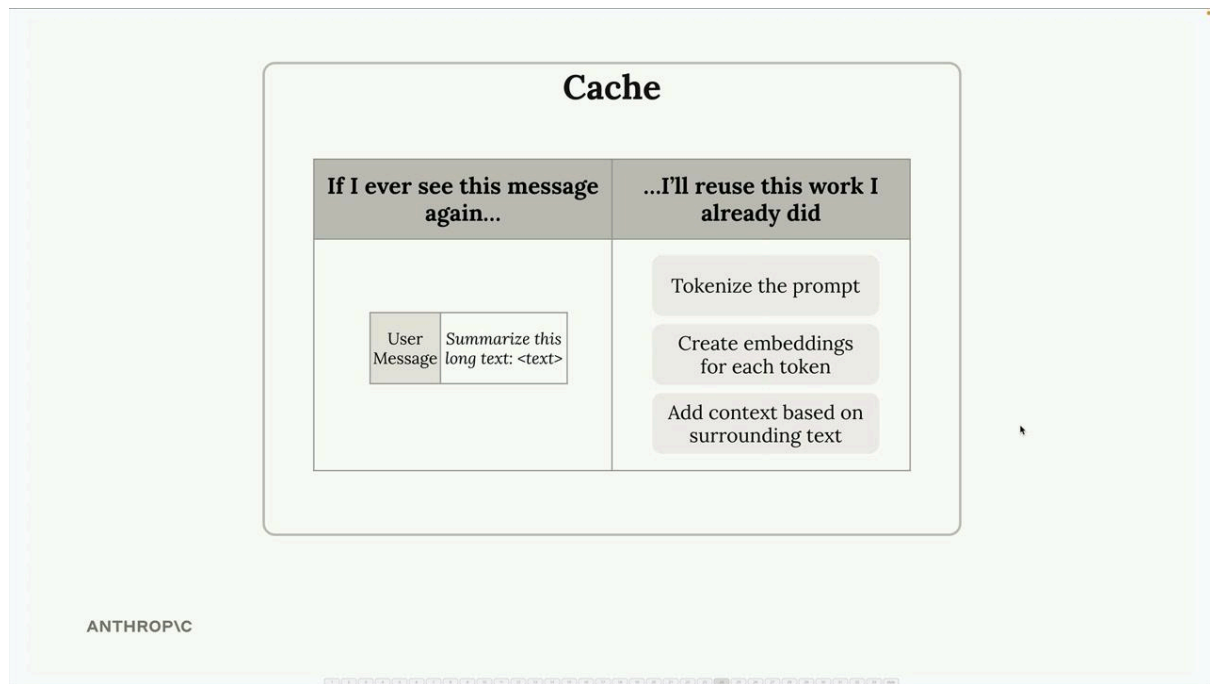


How Prompt Caching Solves This

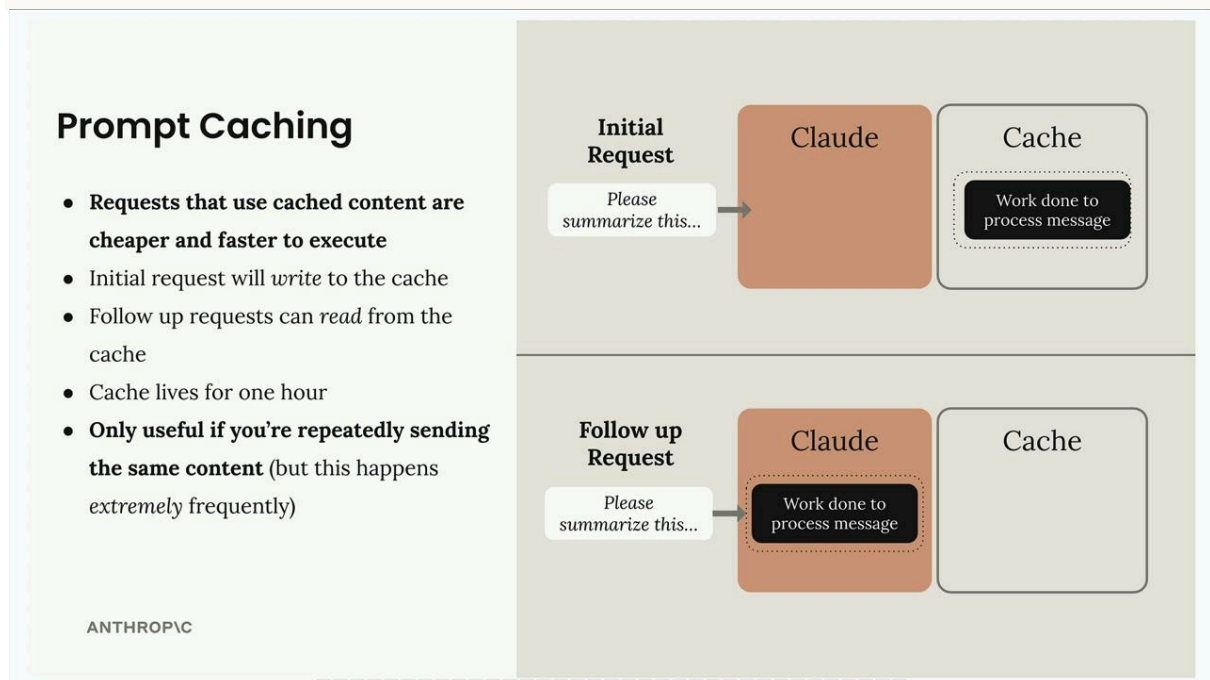
Prompt caching changes this workflow by saving the preprocessing work instead of discarding it:



When you make an initial request, Claude performs all the usual preprocessing but stores the results in a cache instead of throwing them away. The cache acts like a lookup table that says "If I ever see this message again, I'll reuse this work I already did."



Key Benefits and Limitations



Prompt caching offers several advantages:

- Faster responses: Requests using cached content execute more quickly
- Lower costs: You pay less for the cached portions of your requests
- Automatic optimization: The initial request writes to the cache, follow-up requests read from it

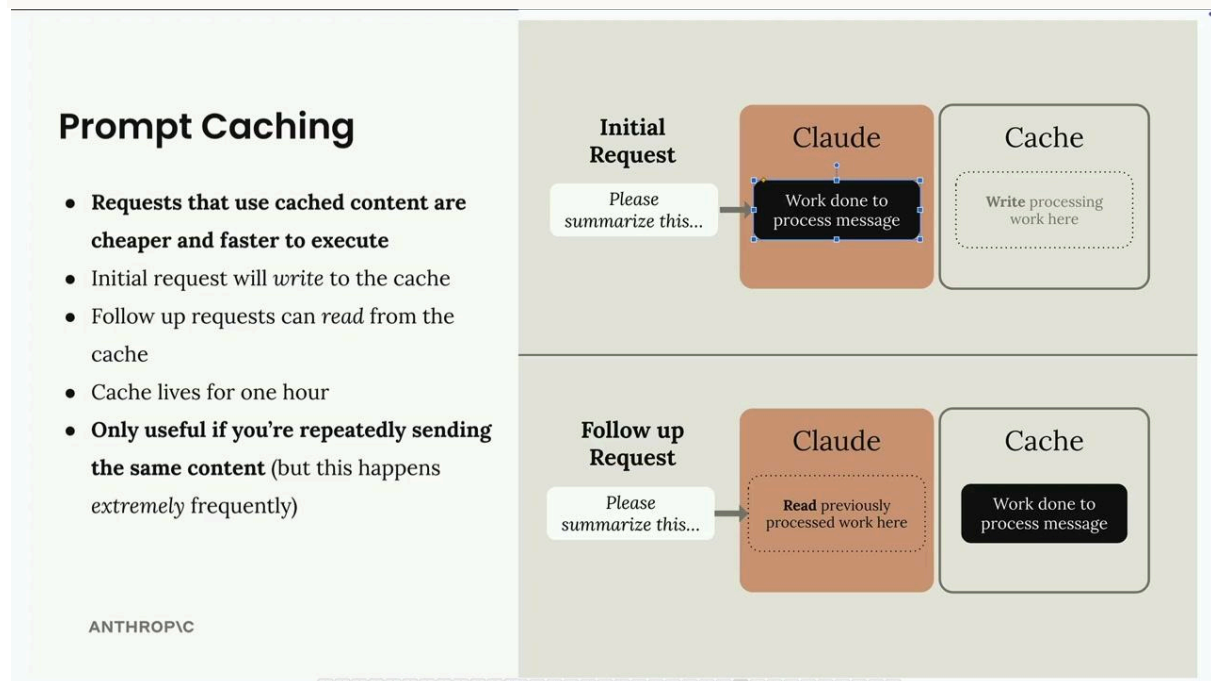
However, there are important limitations to keep in mind:

- Cache duration: Cached content only lives for one hour
- Limited use cases: Only beneficial when you're repeatedly sending the same content
- High frequency requirement: Most effective when the same content appears extremely frequently in your requests

Prompt caching works best for scenarios like document analysis workflows, where you're asking multiple questions about the same large document, or iterative editing tasks where the base content remains constant while you refine specific aspects.

Rules of prompt caching

Prompt caching in Claude works by storing the computational work done on your messages so it can be reused in follow-up requests. This makes subsequent requests both faster and cheaper to execute, but only when you're repeatedly sending identical content.



The process is straightforward: your initial request writes processing work to the cache, and follow-up requests can read from that cache instead of reprocessing the same content. The cache lives for one hour, so this feature is only useful if you're repeatedly sending the same content within that timeframe.

Cache Breakpoints

Caching isn't enabled automatically - you need to manually add cache breakpoints to specific blocks in your messages. Here's how it works:

- Work done on messages is not cached automatically
- You must manually add a 'cache breakpoint' to a block
- Work done for everything before the breakpoint will be cached
- Cache will only be used on follow-up requests if the content up to and including the breakpoint is identical

Cache Breakpoints

- Work done on messages is **not cached automatically**
- We have to manually add a 'cache breakpoint' to a block
- Work done for everything **before** the breakpoint will be cached
- Cache will only be used on follow up requests if the content up to and including the breakpoint is identical

ANTHROPIC

Cache Breakpoint

```
user_message = {
  "role": "user",
  "content": [
    {
      "type": "text",
      "text": "<Long prompt>",
      "cache_control": {
        "type": "ephemeral"
      }
    },
  ],
}
```

To add a cache breakpoint, you need to use the longhand form for writing text blocks instead of the shorthand:

Shorthand for writing a Text Block

```
user_message = {  
  "role": "user",  
  "content": "Hi there!"  
}
```

Longhand for writing a Text Block

```
user_message = {  
  "role": "user",  
  "content": [  
    {  
      "type": "text",  
      "text": "<Long prompt>",  
      "cache_control": {  
        "type": "ephemeral"  
      }  
    },  
  ],  
}
```

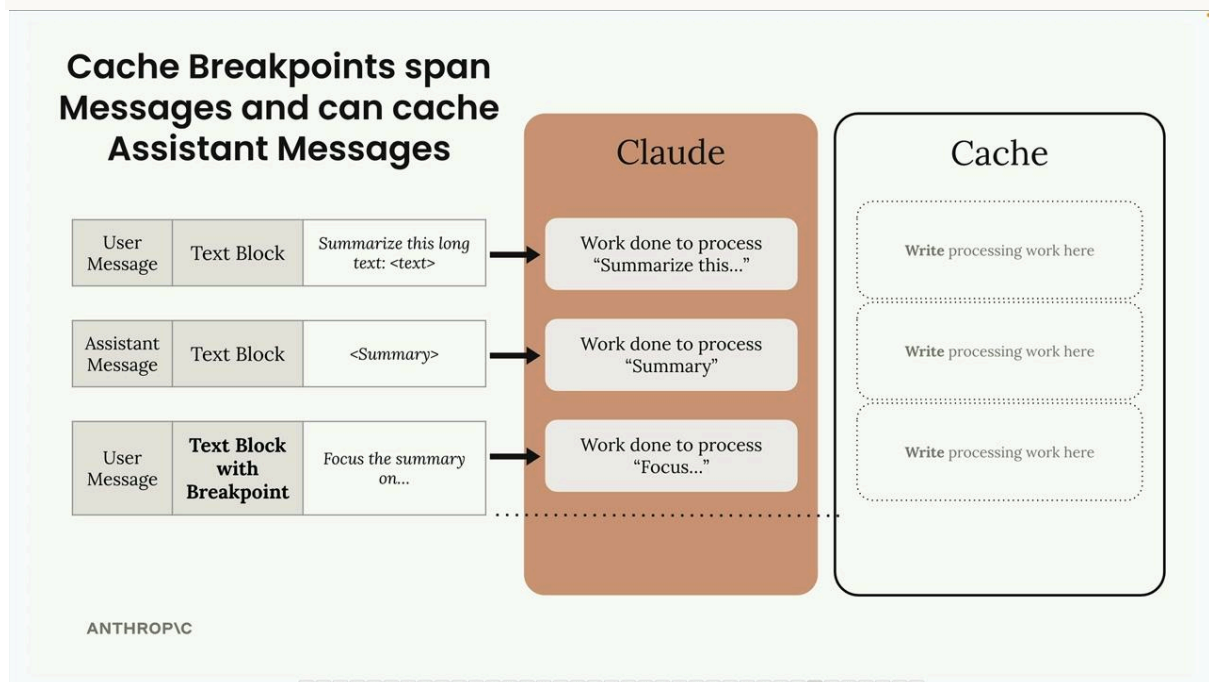
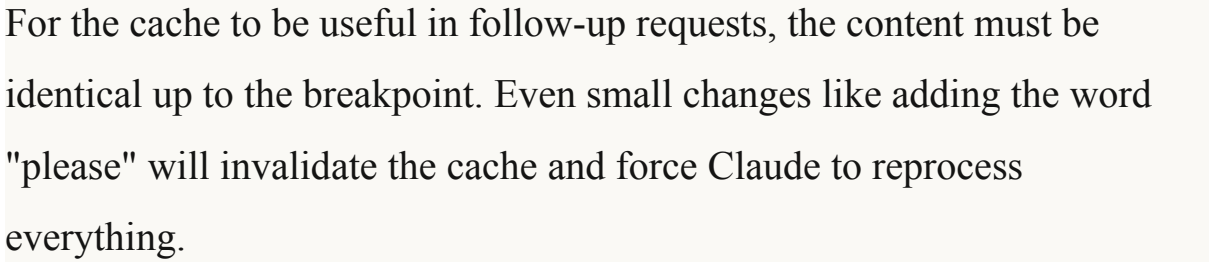
Use longhand form when adding a cache breakpoint

ANTHROPIC

The shorthand form doesn't provide a place to add the cache control field, so you must use the expanded format with the **cache_control** field set to **{"type": "ephemeral"}**.

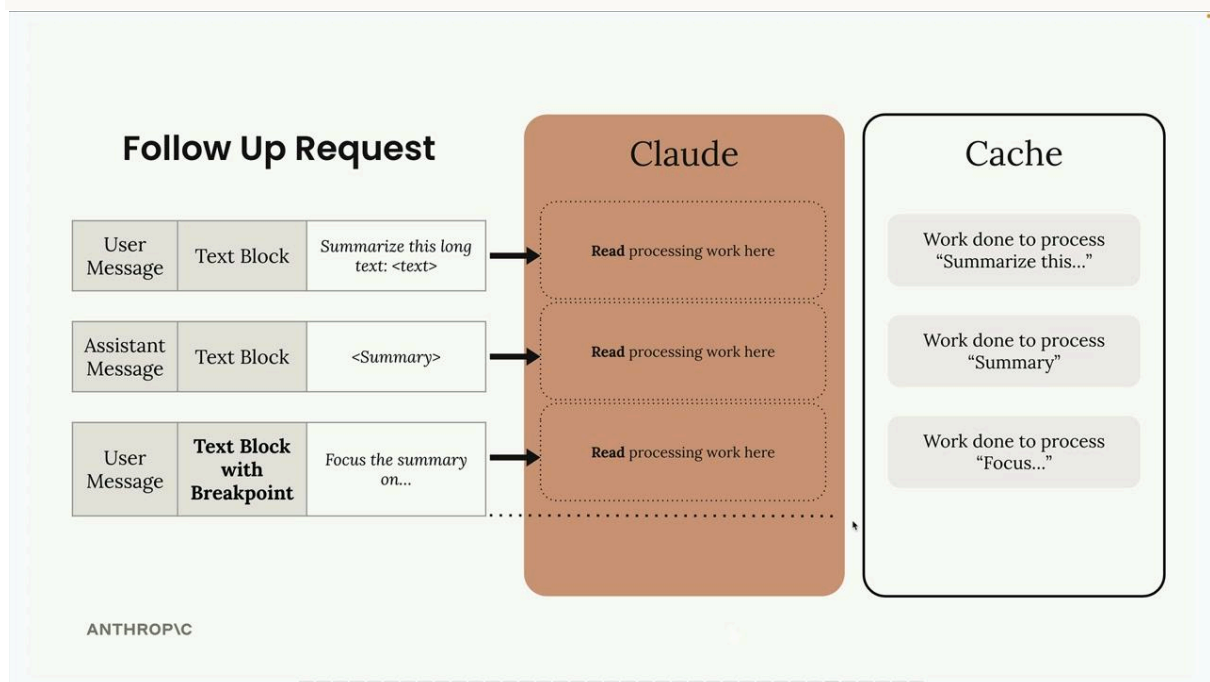
How Cache Breakpoints Work

When you place a cache breakpoint in a message, Claude caches all the processing work up to and including that breakpoint. Content after the breakpoint is processed normally without caching.



Cross-Message Caching

Cache breakpoints can span across multiple messages and message types. If you place a breakpoint in a later message, all previous messages (user, assistant, etc.) will be included in the cached content.



This is particularly useful for conversations where you want to cache the entire context up to a certain point.

System Prompts and Tools

You're not limited to text blocks - cache breakpoints can be added to:

- System prompts
- Tool definitions
- Image blocks

- Tool use and tool result blocks

Breakpoint Location

- Not restricted to text blocks!
- Can add cache breakpoints to system prompts and tool definitions
- Useful when you have a long system prompt or a long list of tools
- **These are the most common caching opportunities since they rarely change between requests!**

ANTHROPIC

Tool definitions will be cached

```
tools = [
  add_duration_to_datetime_schema,
  get_current_datetime_schema,
  {
    "name": "set_reminder",
    "description": "Sets a reminder...",
    "input_schema": { ... },
    "cache_control": {"type": "ephemeral"}
  }
]
```

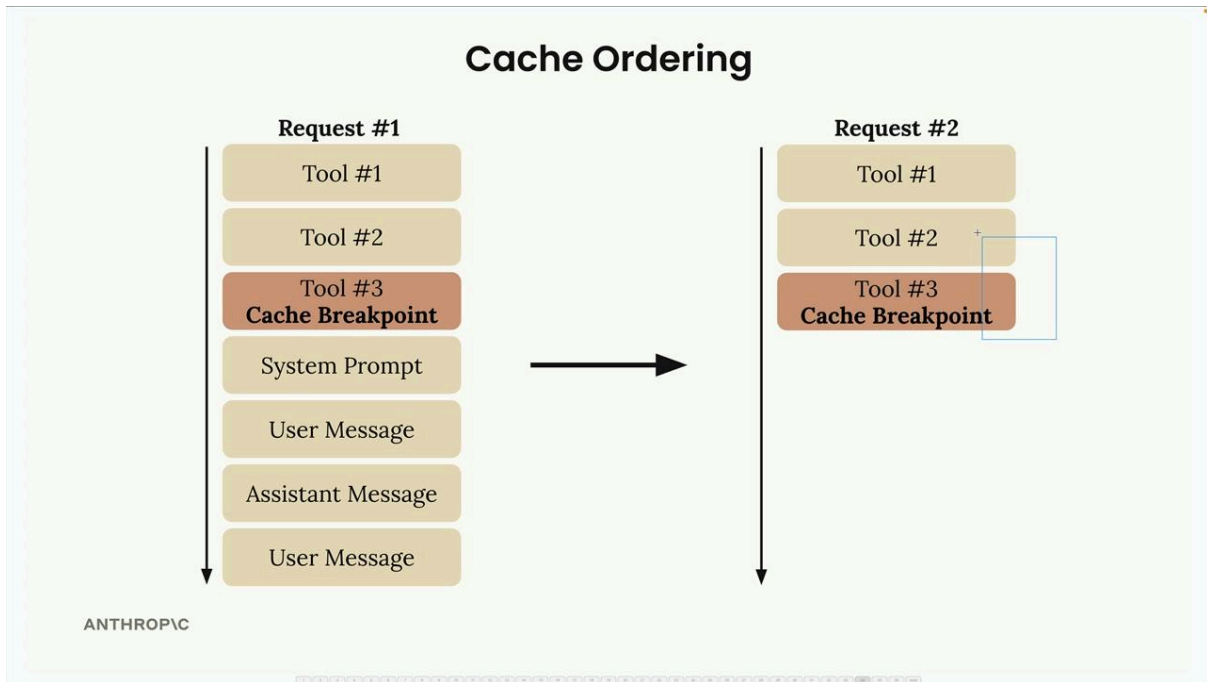
System prompt will be cached

```
system = [
  {
    "type": "text",
    "text": "You are a senior software...",
    "cache_control": {"type": "ephemeral"}
  }
]
```

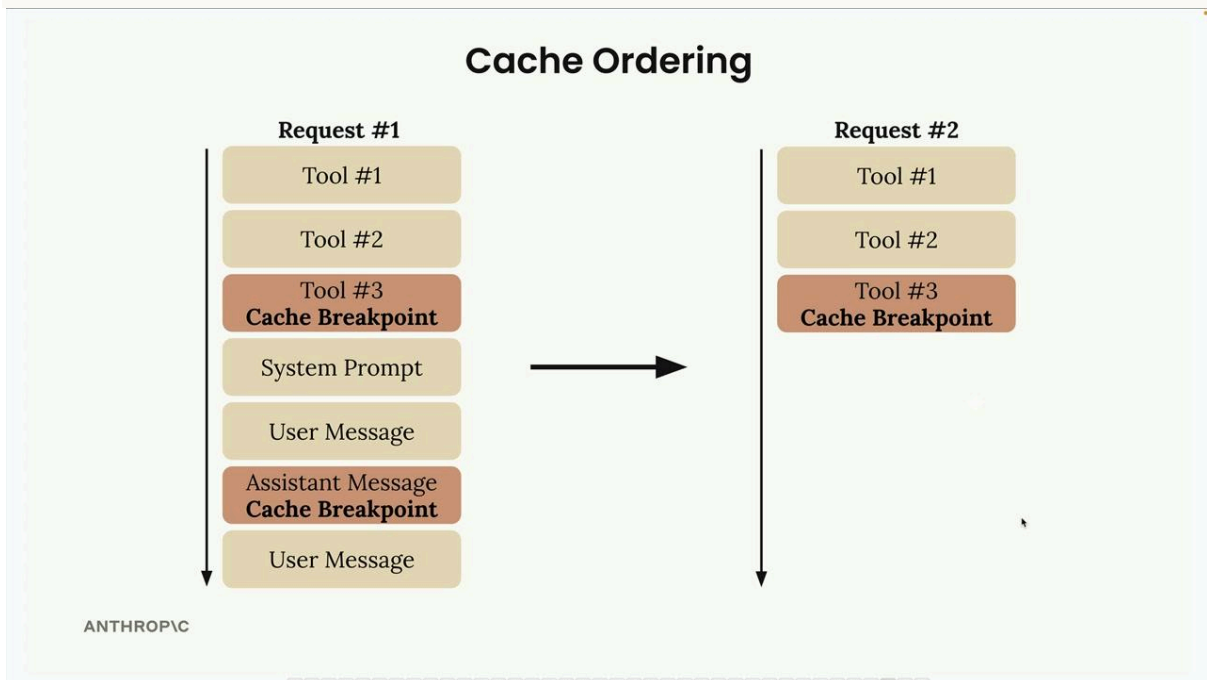
System prompts and tool definitions are excellent candidates for caching since they rarely change between requests. This is often where you'll get the most benefit from prompt caching.

Cache Ordering

Behind the scenes, Claude processes your request components in a specific order: tools first, then system prompt, then messages. Understanding this order helps you place breakpoints effectively.



You can add up to four cache breakpoints total. For example, you might cache your tools, then add another breakpoint partway through your conversation history. This gives you flexibility in what gets cached when different parts of your request change.



Minimum Content Length

There's a minimum threshold for caching: content must be at least 1024 tokens long to be cached. This is the sum of all messages and blocks you're trying to cache, not individual blocks.

Minimum Content Length

- Content must be at least 1024 tokens long to be cached (sum of all messages/blocks you're trying to cache)

ANTHROPIC

"Hi there!" isn't 1024 tokens long - not written to cache

```
conversation = [{
  "role": "user",
  "content": [
    {
      "type": "text",
      "text": "Hi there!",
      "cache_control": {
        "type": "ephemeral"
      }
    }
  ],
}]
```

"Hi there!" duplicated 500 times is >1024 tokens. Cached!

```
conversation = [{
  "role": "user",
  "content": [
    { "type": "text", "text": "Hi there!" },
    { "type": "text", "text": "Hi there!" },
    { "type": "text", "text": "Hi there!" },
    # repeat 500 times...
    {
      "type": "text",
      "text": "Hi there!",
      "cache_control": {
        "type": "ephemeral"
      }
    }
  ],
}]
```

A simple "Hi there!" message won't meet this threshold, but if you duplicate that content 500 times (or have a genuinely long prompt), it will exceed 1024 tokens and be eligible for caching.

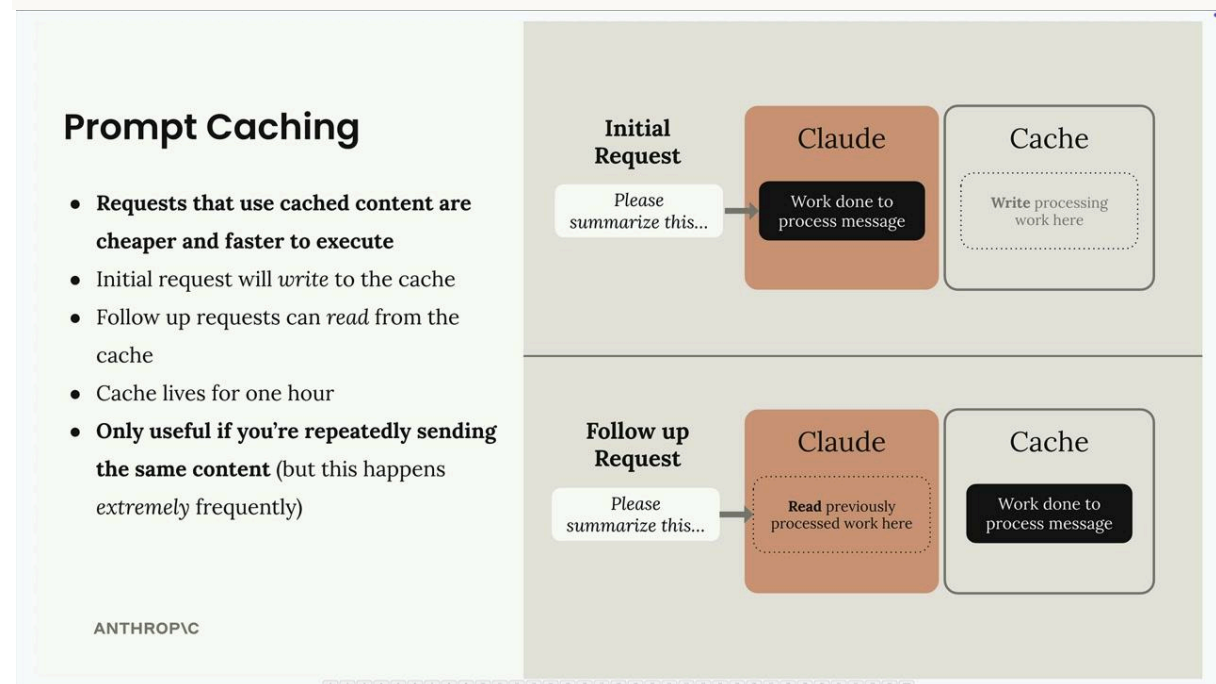
The key to effective prompt caching is identifying which parts of your requests stay consistent across multiple calls and placing breakpoints strategically to maximize reuse while minimizing cache invalidation.

Prompt caching in action

Downloads

003_caching.ipynb

Prompt caching is a powerful optimization feature that makes your API requests both faster and cheaper when you're repeatedly sending the same content to Claude. Let's explore how to implement it effectively in your applications.



How Prompt Caching Works

When you enable prompt caching, the first request writes content to a cache that lives for one hour. Follow-up requests can then read from this

cache instead of processing the same content again. This is particularly valuable when you're sending:

- Large system prompts (like a 6K token coding assistant prompt)
- Complex tool schemas (around 1.7K tokens for multiple tools)
- Repeated message content

The key insight is that caching only helps if you're repeatedly sending identical content - but in many applications, this happens extremely frequently.

Setting Up Tool Schema Caching

To cache your tool schemas, you need to add a cache control field to the last tool in your list. Here's the proper way to do it without modifying your original tool definitions:

```
if tools:
    tools_clone = tools.copy()
    last_tool = tools_clone[-1].copy()
    last_tool["cache_control"] = {"type": "ephemeral"}
    tools_clone[-1] = last_tool

    params["tools"] = tools_clone
```

This approach creates copies of both the tools list and the last tool schema before adding the cache control field. While you could directly modify `tools[-1]["cache_control"]`, the copying approach prevents issues if you later reorder your tools.

System Prompt Caching

For system prompts, you need to structure them as a text block with cache control:

```
if system:
    params["system"] = [
        {
            "type": "text",
            "text": system,
            "cache_control": {"type": "ephemeral"}
        }
    ]
```

This converts your system prompt from a simple string into a structured format that supports caching.

Understanding Cache Behavior

When you run requests with caching enabled, you'll see different usage patterns in the response:

- First request: **cache_creation_input_tokens=1772** - Claude writes to cache
- Follow-up requests: **cache_read_input_tokens=1772** - Claude reads from cache
- Changed content: New cache creation tokens appear

The cache is extremely sensitive - changing even a single character in your tools or system prompt invalidates the entire cache for that component.

Cache Ordering and Breakpoints

You can set multiple cache breakpoints in a single request. The order matters:

1. Tools (if provided)
2. System prompt (if provided)
3. Messages

If you change your system prompt but keep the same tools, you'll see a partial cache read (for tools) and a cache write (for the new system prompt). This granular caching means you only pay for processing the parts that actually changed.

Practical Considerations

Prompt caching is most effective when you have:

- Consistent tool schemas across requests
- Stable system prompts
- Applications that make multiple requests with similar context

Remember that the cache only lasts for one hour, so it's designed for applications with relatively frequent API usage rather than long-term storage.

Code execution and the Files API

Downloads

[streaming.csv](#)

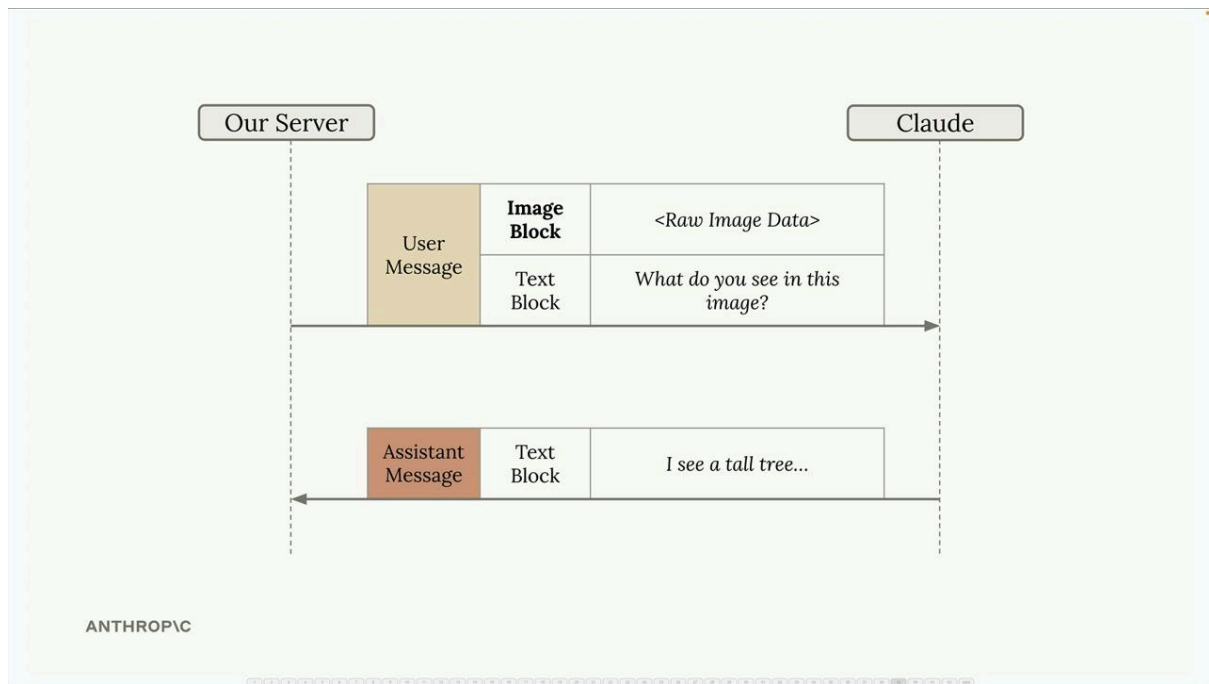
(opens in new tab)

[005_code_execution.ipynb](#)

The Anthropic API offers two powerful features that work exceptionally well together: the Files API and Code Execution. While they might seem separate at first, combining them opens up some really interesting possibilities for delegating complex tasks to Claude.

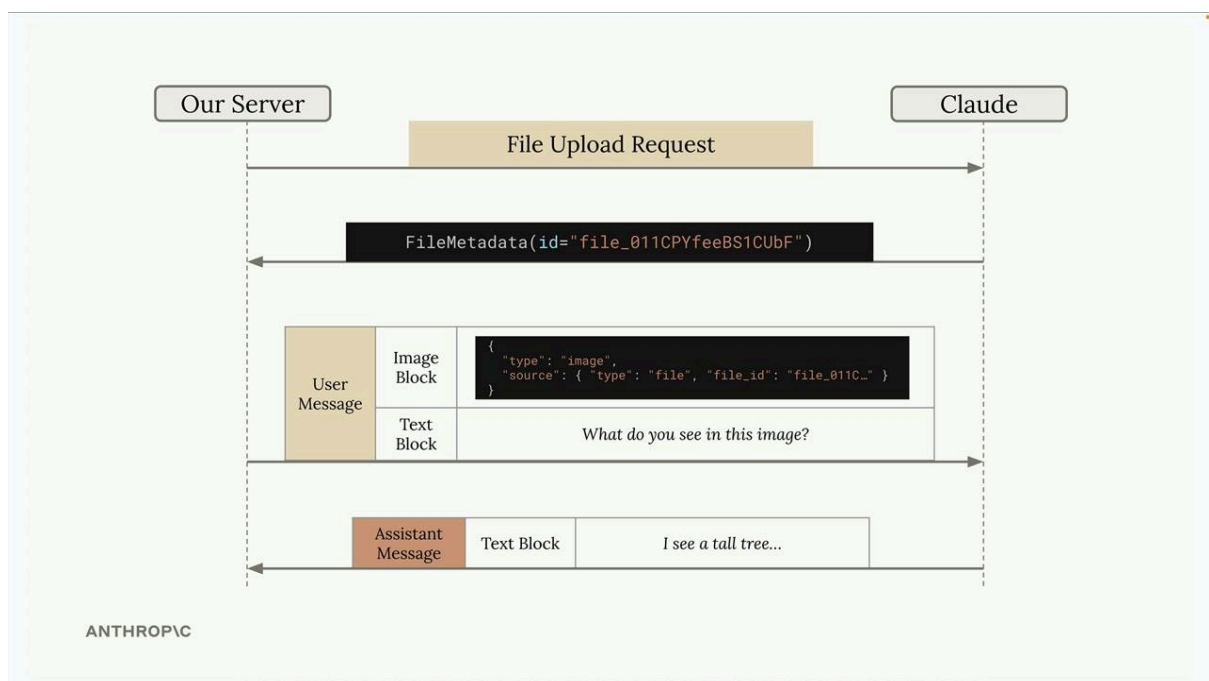
Files API

The Files API provides an alternative way to handle file uploads. Instead of encoding images or PDFs directly in your messages as base64 data, you can upload files ahead of time and reference them later.



Here's how it works:

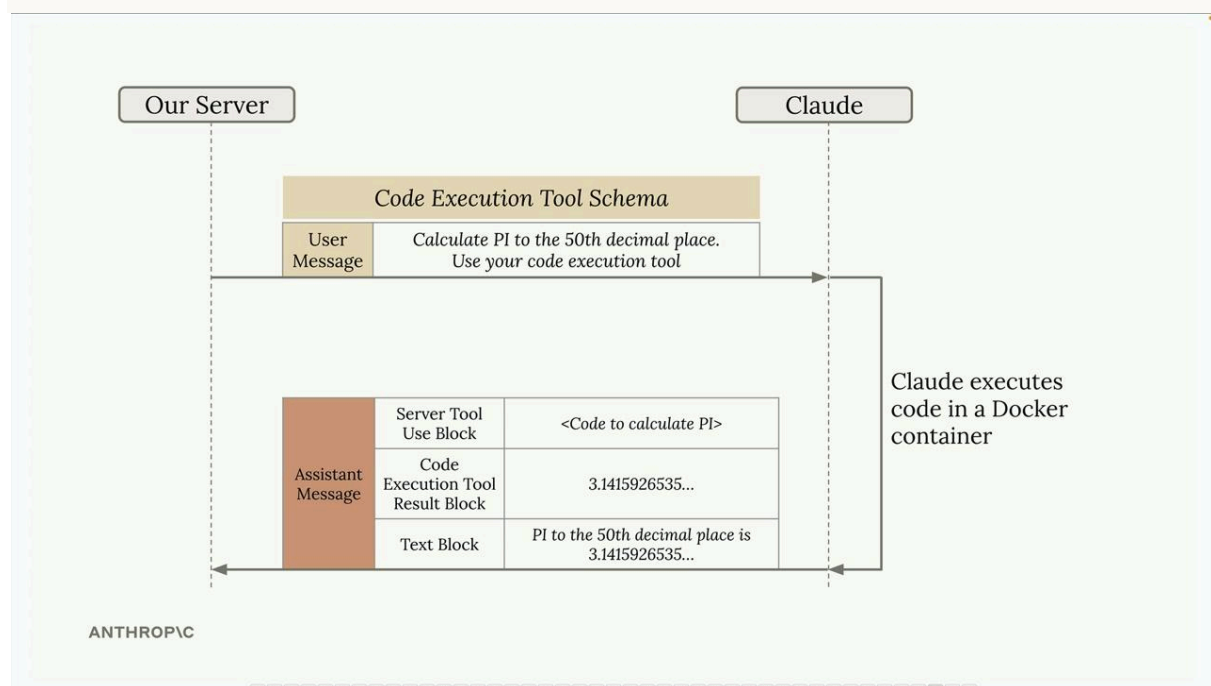
- Upload your file (image, PDF, text, etc.) to Claude using a separate API call
- Receive a file metadata object containing a unique file ID
- Reference that file ID in future messages instead of including raw file data



This approach is particularly useful when you want to reference the same file multiple times or when working with larger files that would be cumbersome to include in every request.

Code Execution Tool

Code execution is a server-based tool that doesn't require you to provide an implementation. You simply include a predefined tool schema in your request, and Claude can optionally execute Python code in an isolated Docker container.



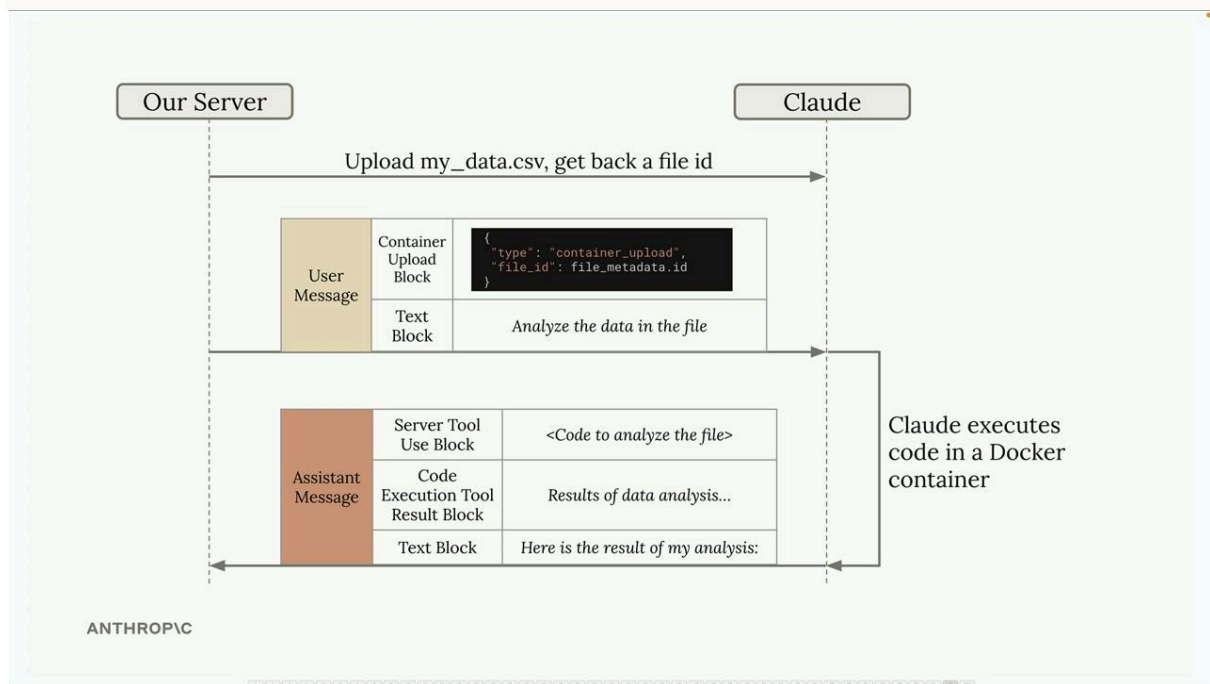
Key characteristics of the code execution environment:

- Runs in an isolated Docker container
- No network access (can't make external API calls)

- Claude can execute code multiple times during a single conversation
- Results are captured and interpreted by Claude for the final response

Combining Files API and Code Execution

The real power comes from using these features together. Since the Docker containers have no network access, the Files API becomes the primary way to get data in and out of the execution environment.



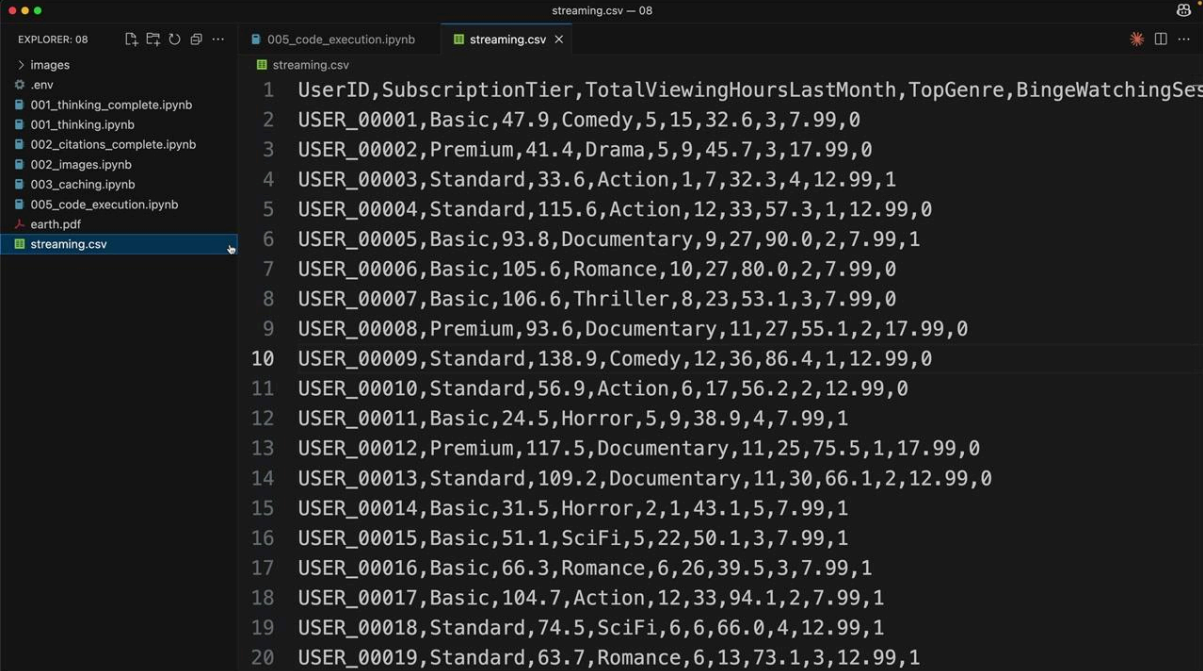
Here's a typical workflow:

1. Upload your data file (like a CSV) using the Files API
2. Include a container upload block in your message with the file ID
3. Ask Claude to analyze the data

4. Claude writes and executes code to process your file
5. Claude can generate outputs (like plots) that you can download

Practical Example

Let's look at a real example using streaming service data. The CSV file contains user information including subscription tiers, viewing habits, and whether they've churned (canceled their subscription).



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a list of files including 'streaming.csv'. The code editor shows the contents of 'streaming.csv', which is a CSV file with 20 rows of user data. The columns are UserID, SubscriptionTier, TotalViewingHoursLastMonth, TopGenre, and BingeWatchingSes. The data is as follows:

UserID	SubscriptionTier	TotalViewingHoursLastMonth	TopGenre	BingeWatchingSes
USER_00001	Basic	47.9	Comedy	5,15,32.6,3,7.99,0
USER_00002	Premium	41.4	Drama	5,9,45.7,3,17.99,0
USER_00003	Standard	33.6	Action	1,7,32.3,4,12.99,1
USER_00004	Standard	115.6	Action	12,33,57.3,1,12.99,0
USER_00005	Basic	93.8	Documentary	9,27,90.0,2,7.99,1
USER_00006	Basic	105.6	Romance	10,27,80.0,2,7.99,0
USER_00007	Basic	106.6	Thriller	8,23,53.1,3,7.99,0
USER_00008	Premium	93.6	Documentary	11,27,55.1,2,17.99,0
USER_00009	Standard	138.9	Comedy	12,36,86.4,1,12.99,0
USER_00010	Standard	56.9	Action	6,17,56.2,2,12.99,0
USER_00011	Basic	24.5	Horror	5,9,38.9,4,7.99,1
USER_00012	Premium	117.5	Documentary	11,25,75.5,1,17.99,0
USER_00013	Standard	109.2	Documentary	11,30,66.1,2,12.99,0
USER_00014	Basic	31.5	Horror	2,1,43.1,5,7.99,1
USER_00015	Basic	51.1	SciFi	5,22,50.1,3,7.99,1
USER_00016	Basic	66.3	Romance	6,26,39.5,3,7.99,1
USER_00017	Basic	104.7	Action	12,33,94.1,2,7.99,1
USER_00018	Standard	74.5	SciFi	6,6,66.0,4,12.99,1
USER_00019	Standard	63.7	Romance	6,13,73.1,3,12.99,1

First, upload the file using a helper function:

```
file_metadata = upload('streaming.csv')
```

Then create a message that includes both the uploaded file and a request for analysis:

```
messages = []  
add_user_message(
```

```

    messages,
    [
        {
            "type": "text",
            "text": """Run a detailed analysis to determine
major drivers of churn.
Your final output should include at least one
detailed plot summarizing your findings."""
        },
        {"type": "container_upload", "file_id":
file_metadata.id},
    ],
)

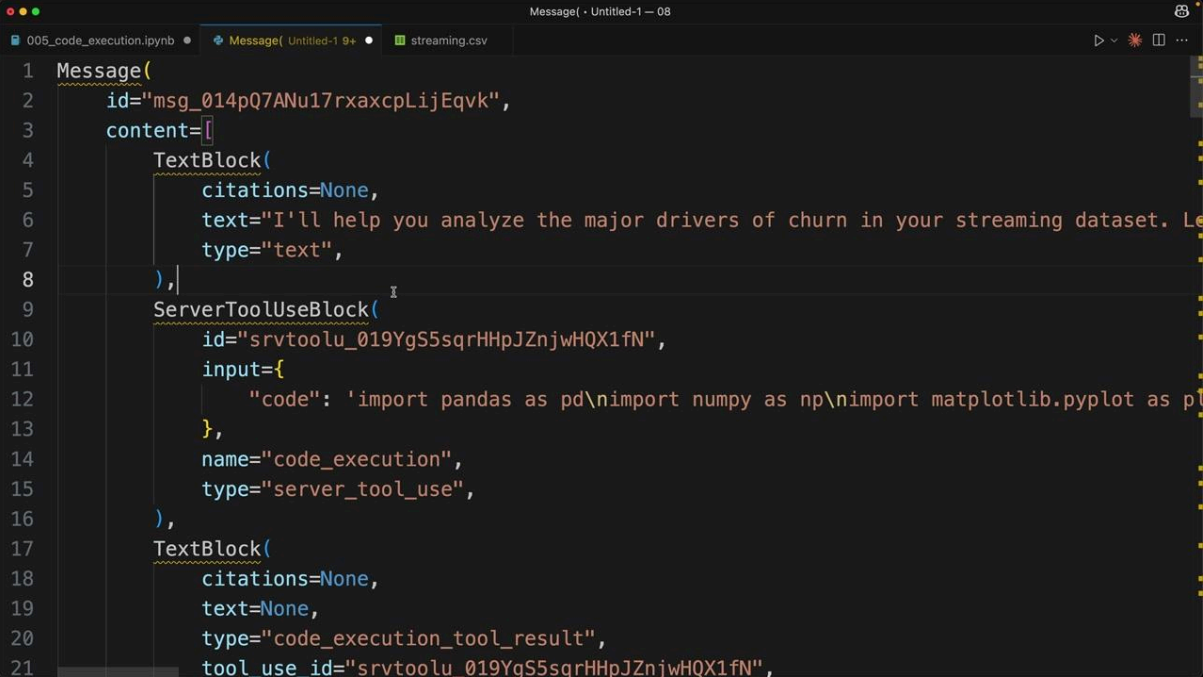
chat(
    messages,
    tools=[{"type": "code_execution_20250522", "name":
"code_execution"}]
)

```

Understanding the Response

When Claude uses code execution, the response contains multiple types of blocks:

- Text blocks - Claude's analysis and explanations
- Server tool use blocks - The actual code Claude decided to run
- Code execution tool result blocks - Output from running the code



```

1 Message(
2     id="msg_014pQ7ANu17rxaxcpLijEqvk",
3     content=[
4         TextBlock(
5             citations=None,
6             text="I'll help you analyze the major drivers of churn in your streaming dataset. Le
7             type="text",
8         ),
9         ServerToolUseBlock(
10            id="srvtoolu_019YgS5sqrHHpJZnjwHQX1fN",
11            input={
12                "code": 'import pandas as pd\nimport numpy as np\nimport matplotlib.pyplot as p
13            },
14            name="code_execution",
15            type="server_tool_use",
16        ),
17        TextBlock(
18            citations=None,
19            text=None,
20            type="code_execution_tool_result",
21            tool_use_id="srvtoolu_019YgS5sqrHHpJZnjwHQX1fN",

```

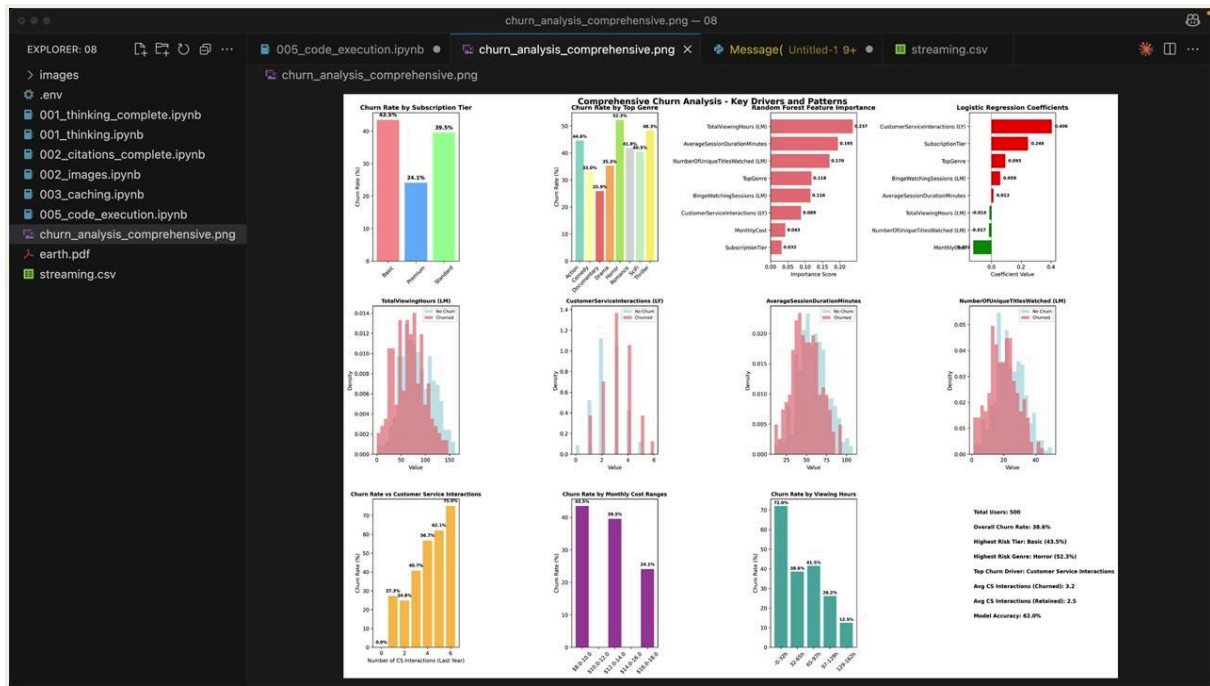
Claude might execute code multiple times during a single response, iteratively building up its analysis. Each execution cycle includes the code and its results.

Downloading Generated Files

One of the most powerful features is Claude's ability to generate files (like plots or reports) and make them available for download. When Claude creates a visualization, it gets stored in the container and you can download it using the Files API.

Look for blocks with **type: "code_execution_output"** in the response - these contain file IDs for generated content:

```
download_file("file_id_from_response")
```



The result is a comprehensive analysis with professional visualizations that would have taken significant manual coding to produce.

Beyond Data Analysis

While data analysis is a natural fit, the combination of Files API and code execution opens up many possibilities:

- Image processing and manipulation
- Document parsing and transformation
- Mathematical computations and modeling
- Report generation with custom formatting

The key is that you can delegate complex, computational tasks to Claude while maintaining control over the inputs and outputs through the Files

API. This creates a powerful workflow where Claude becomes your coding assistant that can actually execute and iterate on solutions.